MINE 4201 - Laboratorio 2: Filtrado por Contenido y Estrategias de Embedding

En este laboratorio exploraremos los sistemas de recomendación basados en contenido, utilizando diferentes estrategias de representación vectorial (embeddings) para describir los ítems:

1. **TF-IDF** (Term Frequency – Inverse Document Frequency)
2. **Word2Vec** (Representaciones densas aprendidas)

Al finalizar, el estudiante será capaz de:

- Construir una matriz documento-término y calcular TF-IDF
- Aplicar selección de características para mejorar la representación
- Entrenar y evaluar un modelo de clasificación para recomendación por contenido
- Comprender el modelo Word2Vec y sus arquitecturas (Skip-gram, CBOW)
- Comparar las representaciones TF-IDF vs Word2Vec para filtrado por contenido
- Reflexionar sobre cómo estas estrategias de embedding se conectan con el Filtrado Colaborativo

---

Contexto: Filtrado por Contenido vs Filtrado Colaborativo

En los sistemas de recomendación existen dos grandes familias de enfoques:

Enfoque	Descripción	Datos que utiliza
Filtrado por Contenido	Recomienda ítems similares a los que el usuario ha preferido, basándose en las **características del ítem** (género, descripción, conceptos).	Atributos / features de los ítems
Filtrado Colaborativo	Recomienda ítems que usuarios similares han preferido, sin necesitar características del ítem.	Matriz de interacciones usuario-ítem

¿Dónde entran los Embeddings?

Las estrategias de embedding (TF-IDF, Word2Vec, etc.) pueden utilizarse en **ambos** enfoques:

- **En filtrado por contenido:** se usan para representar las características de los ítems en un espacio vectorial continuo. Es lo que haremos en la primera parte de este laboratorio.
- **En filtrado colaborativo:** los embeddings pueden representar usuarios e ítems en un espacio latente compartido (e.g., factorización de matrices, embeddings neurales).

Modelos como Word2Vec han inspirado técnicas como **Item2Vec**, donde se trata la secuencia de ítems consumidos por un usuario como una "oración" y se aprenden embeddings de ítems.

En este laboratorio nos centraremos en el **filtrado por contenido** usando TF-IDF y exploraremos Word2Vec como estrategia alternativa de representación.

Los sistemas de recomendación basados en contenido filtran contenido basado en la representación de items y el perfil del usuario. En este laboratorio trabajaremos con un conjunto de datos del sitio web LibraryThing.

Preparación del entorno

Instale las librerias que vamos a utilizar e importelas en el ambiente de ejecución

```
!pip install scikit-learn
!pip install pandas
!pip install seaborn
```

```
Collecting scikit-learn
  Using cached scikit_learn-1.8.0-cp311-cp311-
macosx_12_0_arm64.whl.metadata (11 kB)
Collecting numpy>=1.24.1 (from scikit-learn)
  Downloading numpy-2.4.2-cp311-cp311-macosx_14_0_arm64.whl.metadata
(6.6 kB)
Collecting scipy>=1.10.0 (from scikit-learn)
  Downloading scipy-1.17.1-cp311-cp311-macosx_14_0_arm64.whl.metadata
(62 kB)
Collecting joblib>=1.3.0 (from scikit-learn)
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.2.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Using cached scikit_learn-1.8.0-cp311-cp311-macosx_12_0_arm64.whl (8.1
MB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading numpy-2.4.2-cp311-cp311-macosx_14_0_arm64.whl (5.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.5/5.5 MB 46.7 MB/s eta
0:00:00
acosx_14_0_arm64.whl (20.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 20.3/20.3 MB 63.6 MB/s eta
0:00:00a 0:00:01
py, joblib, scipy, scikit-learn
Successfully installed joblib-1.5.3 numpy-2.4.2 scikit-learn-1.8.0
scipy-1.17.1 threadpoolctl-3.6.0

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Collecting pandas
  Downloading pandas-3.0.1-cp311-cp311-macosx_11_0_arm64.whl.metadata
(79 kB)
```

```
Requirement already satisfied: numpy>=1.26.0 in
./.venv/lib/python3.11/site-packages (from pandas) (2.4.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
./.venv/lib/python3.11/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in
./.venv/lib/python3.11/site-packages (from python-dateutil>=2.8.2-
>pandas) (1.17.0)
Downloading pandas-3.0.1-cp311-cp311-macosx_11_0_arm64.whl (9.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.9/9.9 MB 60.0 MB/s eta
0:00:00
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in
./.venv/lib/python3.11/site-packages (from seaborn) (2.4.2)
Requirement already satisfied: pandas>=1.2 in
./.venv/lib/python3.11/site-packages (from seaborn) (3.0.1)
Collecting matplotlib!=3.6.1,>=3.4 (from seaborn)
  Using cached matplotlib-3.10.8-cp311-cp311-
macosx_11_0_arm64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached contourpy-1.3.3-cp311-cp311-
macosx_11_0_arm64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached fonttools-4.61.1-cp311-cp311-
macosx_10_9_universal2.whl.metadata (114 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached kiwisolver-1.4.9-cp311-cp311-
macosx_11_0_arm64.whl.metadata (6.3 kB)
Requirement already satisfied: packaging>=20.0 in
./.venv/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (26.0)
Collecting pillow>=8 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached pillow-12.1.1-cp311-cp311-
macosx_11_0_arm64.whl.metadata (8.8 kB)
Collecting pyparsing>=3 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in
./.venv/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in
./.venv/lib/python3.11/site-packages (from python-dateutil>=2.7-
>matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached matplotlib-3.10.8-cp311-cp311-macosx_11_0_arm64.whl (8.1
MB)
```

```
Using cached contourpy-1.3.3-cp311-cp311-macosx_11_0_arm64.whl (270
kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.61.1-cp311-cp311-macosx_10_9_universal2.whl
(2.9 MB)
Using cached kiwisolver-1.4.9-cp311-cp311-macosx_11_0_arm64.whl (65
kB)
Using cached pillow-12.1.1-cp311-cp311-macosx_11_0_arm64.whl (4.7 MB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, kiwisolver,
fonttools, cycler, contourpy, matplotlib, seaborn
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.61.1
kiwisolver-1.4.9 matplotlib-3.10.8 pillow-12.1.1 pyparsing-3.3.2
seaborn-0.13.2

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
```

```
import os
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import random
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.feature_selection import chi2
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix,
precision_recall_fscore_support
```

Carga de archivos

Copie el archivo del dataset en el entorno colab en la pestaña files (la carpeta en el menú de la izquierda), carguemos el csv en un dataframe de pandas y revisemos su contenido.

```
if not os.path.exists('DB-BOOK-content.csv') :
  raise ValueError('El archivo DB-BOOK-content.csv no fue encontrado
en el path')
else:
  print("Los archivos han sido cargados")
```

```
Los archivos han sido cargados
```

```
df_dbbook=pd.read_csv('DB-BOOK-content.csv', sep=';')
df_dbbook
```

```
         DBbook_userID                                name
DBbook_ItemID  \
0                    1  Contingency, Irony, and Solidarity
508
1                    1  Contingency, Irony, and Solidarity
508
2                    1  Contingency, Irony, and Solidarity
508
3                    1  Contingency, Irony, and Solidarity
508
4                    1  Contingency, Irony, and Solidarity
508
...                ...                                 ...
...
1439449           7255              The Catcher in the Rye
8010
1439450           7255              The Catcher in the Rye
8010
1439451           7255              The Catcher in the Rye
8010
1439452           7255              The Catcher in the Rye
8010
1439453           7255              The Catcher in the Rye
8010

         featureID                     featureShortname  rate
0             1066                 /Category:1989_books     5
1             2300                       /Richard_Rorty     5
2             2301  /Category:Works_about_Marcel_Proust     5
3             2302     /Category:Books_by_Richard_Rorty     5
4             2303          /Category:Books_about_irony     5
...            ...                                  ...   ...
1439449       3322    /Category:American_bildungsromans     3
1439450       6746            /Category:1949_in_fiction     3
1439451       9011                      /J._D._Salinger     3
1439452      10155   /Category:Novels_by_J._D._Salinger     3
1439453      13857                /NovelsByJ.D.Salinger     3

[1439454 rows x 6 columns]
```

Este dataset tiene un formato similar al que manejamos el taller pasado. Tiene una columna con el id del usuario, otra con el id del item y un rating.

Cada rating esta presente varias veces, una por cada característica de los libros, revisemos por ejemplo las características del libro con ID 8010.

Se selecciona del dataframe las columnas name, featureID y featureShortname, la instruccion drop_duplicates nos deja solamente las columnas que no

Nota: Para saber más de indexación de dataframes de pandas utilizando .loc hay una explicación en la documentación

```
df_dbbook.loc[df_dbbook.DBbook_ItemID==8010,
['DBbook_ItemID','name','featureID','featureShortname']].drop_duplicat
es()
```

```
      DBbook_ItemID                     name  featureID  \
996            8010  The Catcher in the Rye          3
997            8010  The Catcher in the Rye          4
998            8010  The Catcher in the Rye          5
999            8010  The Catcher in the Rye          6
1000           8010  The Catcher in the Rye          7
1001           8010  The Catcher in the Rye         11
1002           8010  The Catcher in the Rye         13
1003           8010  The Catcher in the Rye        228
1004           8010  The Catcher in the Rye        233
1005           8010  The Catcher in the Rye        277
1006           8010  The Catcher in the Rye        626
1007           8010  The Catcher in the Rye       1378
1008           8010  The Catcher in the Rye       2394
1009           8010  The Catcher in the Rye       2402
1010           8010  The Catcher in the Rye       3322
1011           8010  The Catcher in the Rye       6746
1012           8010  The Catcher in the Rye       9011
1013           8010  The Catcher in the Rye      10155
1014           8010  The Catcher in the Rye      13857

                              featureShortname
996                    /Communication100033020
997                          /Fiction106367107
998               /LiteraryComposition106364329
999                          /Writing106362953
1000             /WrittenCommunication106349220
1001                           /Novel106367879
1002                     /Abstraction100002137
1003                              /DebutNovels
1004                    /Category:Debut_novels
1005  /Category:Little,_Brown_and_Company_books
1006      /Category:Novels_set_in_New_York_City
1007       /Category:Novels_set_in_Pennsylvania
1008                              /1951Novels
1009                     /Category:1951_novels
1010          /Category:American_bildungsromans
1011                  /Category:1949_in_fiction
1012                           /J._D._Salinger
1013         /Category:Novels_by_J._D._Salinger
1014                     /NovelsByJ.D.Salinger
```

Cada libro tiene features que fueron extraidos de DBpedia. Dbpedia es una iniciativa para construir una representación de conceptos y relaciones mediante ontologías utilizando la información depositada en wikipedia. Más adelante vamos a utilizar DBPedia para otros talleres que aprovechan la información de la ontología de DBPedia, por ahora lo que tenemos es una representación de conjunto de palabras (o conceptos) para describir un item.

Creación de matriz documento-termino

En las siguientes líneas vamos a crear la matriz documento término, el primer paso es obtener en un dataframe los libros, los conceptos, y los conceptos únicos por libro.

```
df_libros=df_dbbook.loc[:,['DBbook_ItemID','name']].drop_duplicates()
df_libros
```

```
         DBbook_ItemID                                      name
0                  508       Contingency, Irony, and Solidarity
8                  802                     The Solitaire Mystery
26                 889                                Wise Blood
49                1367                               The Bell Jar
72                1736  The Structure of Scientific Revolutions
...                ...                                       ...
1434966           2860                           Practical Magic
1436627           3838  Destiny: A Chronicle of Deaths Foretold
1437751           5779                         The Road to Omaha
1438719           3485                            Kamikaze Girls
1438753           3963                           Queen of Demons

[6164 rows x 2 columns]
```

```
df_conceptos=df_dbbook.loc[:,
['featureID','featureShortname']].drop_duplicates()
df_conceptos
```

```
         featureID                     featureShortname
0             1066                 /Category:1989_books
1             2300                       /Richard_Rorty
2             2301  /Category:Works_about_Marcel_Proust
3             2302     /Category:Books_by_Richard_Rorty
4             2303          /Category:Books_about_irony
...            ...                                  ...
1438749       8790     /Category:Adventure_comedy_films
1438750       8791       /Category:Fantasy-comedy_films
1438752       8793        /Category:2004_Japanese_films
1438766       9393                  /NovelsByDavidDrake
1438767       9394      /Category:Novels_by_David_Drake

[12246 rows x 2 columns]
```

```
df_libros_concepto=df_dbbook.loc[:,
['DBbook_ItemID','name','featureID','featureShortname']].drop_duplicat
```

```
es()
df_libros_concepto
```

```
         DBbook_ItemID                                name  featureID
\
0                  508  Contingency, Irony, and Solidarity       1066

1                  508  Contingency, Irony, and Solidarity       2300

2                  508  Contingency, Irony, and Solidarity       2301

3                  508  Contingency, Irony, and Solidarity       2302

4                  508  Contingency, Irony, and Solidarity       2303

...                 ...                                 ...        ...

1438763           3963                      Queen of Demons        790

1438764           3963                      Queen of Demons        793

1438765           3963                      Queen of Demons       4006

1438766           3963                      Queen of Demons       9393

1438767           3963                      Queen of Demons       9394


                            featureShortname
0                      /Category:1989_books
1                            /Richard_Rorty
2        /Category:Works_about_Marcel_Proust
3           /Category:Books_by_Richard_Rorty
4                /Category:Books_about_irony
...                                      ...
1438763                           /1998Novels
1438764                /Category:1998_novels
1438765                          /David_Drake
1438766                   /NovelsByDavidDrake
1438767      /Category:Novels_by_David_Drake

[102803 rows x 4 columns]
```

Se quiere obtener una martiz que tenga como filas cada uno de los libros, y como columnas cada uno de los conceptos, y en cada coordenada un 1 si esta presente el concepto y 0 de lo contrario. A esta operación se le conoce como pivot de una tabla.

La función pivot toma tres parámetros: la columna del dataframe original que va a tomar el índice de las filas del nuevo dataframe (index), la columna del dataframe original mediante la cual se van a generar las columnas (columns) y values los valores con los cuales se va a llenar el

dataframe, en este caso vamos a dejar momentaneamente el id del feature para indicar que existe.

Finalmente aplicamos la función notna para modificar uno a uno los elementos de la matriz

```
df_matriz_libros_concepto = (
    df_libros_concepto[['DBbook_ItemID', 'featureID']]
      .drop_duplicates()
      .pivot(index='DBbook_ItemID', columns='featureID',
values='featureID')
      .notna()              # vectorized (fast)
      .astype('int8')       # 1/0 in int8
)

df_matriz_libros_concepto
```

```
featureID      1      2      3      4      5      6      7      8
9      \
DBbook_ItemID

1                  1      1      1      1      1      1      1      1
1
2                  1      1      1      1      1      1      1      0
1
3                  0      0      1      1      1      1      1      1
0
5                  0      0      1      1      1      1      1      0
0
7                  0      0      1      1      1      1      1      0
0
...              ...    ...    ...    ...    ...    ...    ...    ...
...
8164               0      0      1      1      1      1      1      0
0
8166               0      0      0      0      0      0      0      0
0
8167               0      0      0      0      0      0      0      0
0
8168               0      0      1      1      1      1      1      0
0
8169               0      0      1      1      1      1      1      0
0

featureID      10     ...  13992  13993  13994  13995  13996  13997
13998  \
DBbook_ItemID         ...

1                  1  ...      0      0      0      0      0      0
0
2                  1  ...      0      0      0      0      0      0
```

```
0
3                   0  ...      0      0      0      0      0      0
0
5                   0  ...      0      0      0      0      0      0
0
7                   0  ...      0      0      0      0      0      0
0
...               ...  ...    ...    ...    ...    ...    ...    ...
...
8164                0  ...      0      0      0      0      0      0
0
8166                0  ...      0      0      0      0      0      0
0
8167                0  ...      0      0      0      0      0      0
0
8168                0  ...      0      0      0      0      0      0
0
8169                0  ...      0      0      0      1      1      0
0

featureID      13999  14000  14001
DBbook_ItemID
1                   0      0      0
2                   0      0      0
3                   0      0      0
5                   0      0      0
7                   0      0      0
...               ...    ...    ...
8164                0      0      0
8166                0      0      0
8167                0      0      0
8168                0      0      0
8169                0      0      0

[6164 rows x 12246 columns]
```

```
df_matriz_libros_concepto.shape
```

```
(6164, 12246)
```

En la siguiente celda se está aplicando la función sumatoria por eje al dataframe anterior, por defecto el eje es 0, por lo que generará una Serie (vector) del tamaño de las columnas y por cada columna calculará la sumatoria

```
pd.DataFrame.sum
```

```
<function pandas.core.frame.DataFrame.sum(self, *, axis: 'Axis | None'
= 0, skipna: 'bool' = True, numeric_only: 'bool' = False, min_count:
'int' = 0, **kwargs) -> 'Series'>
```

```
series_suma=df_matriz_libros_concepto.sum()
series_suma
```

```
featureID
1            6
2          225
3         2766
4         2672
5         2674
          ...
13997        1
13998        1
13999        1
14000        1
14001        1
Length: 12246, dtype: int64
```

El objeto series_suma no es un dataframe sino un objeto tipo Series, que es un arreglo. Un dataframe puede ser visto como una concatenación de varios objetos de tipo Series

```
type(series_suma)
```

```
pandas.Series
```

Interprete las siguientes figuras y diga qué quieren decir en términos del número de items y de características asignadas a los items

FIGURA 1. Número de ítems (Libros) por característica

Cada punto en la figura representa una característica (feature) y el número de ítems (Libros) a los que se le asignó esa característica.

La figura indica que el conjunto de datos tiene un gran número de características que están asignadas a muy pocos items, mientras que solo unas pocas características aparecen asociadas a una gran cantidad de items. En otras palabras, la mayoría de las características son poco frecuentes y solo un pequeño grupo de ellas es muy común entre los items.

- Los items no comparten todas las características por igual
- Hay características muy discriminativas, porque aparecen en pocos libros
- Hay características poco discriminativas, porque aparecen en casi todos

Esto puede ser de ayuda ya qué:

- Una característica muy común aporta poca diferenciación
- Una característica rara puede ayudar más a distinguir items

```
sns.scatterplot(x=range(0,len(series_suma)) ,y=series_suma.sort_values
() )
```

```
<Axes: ylabel='None'>
```

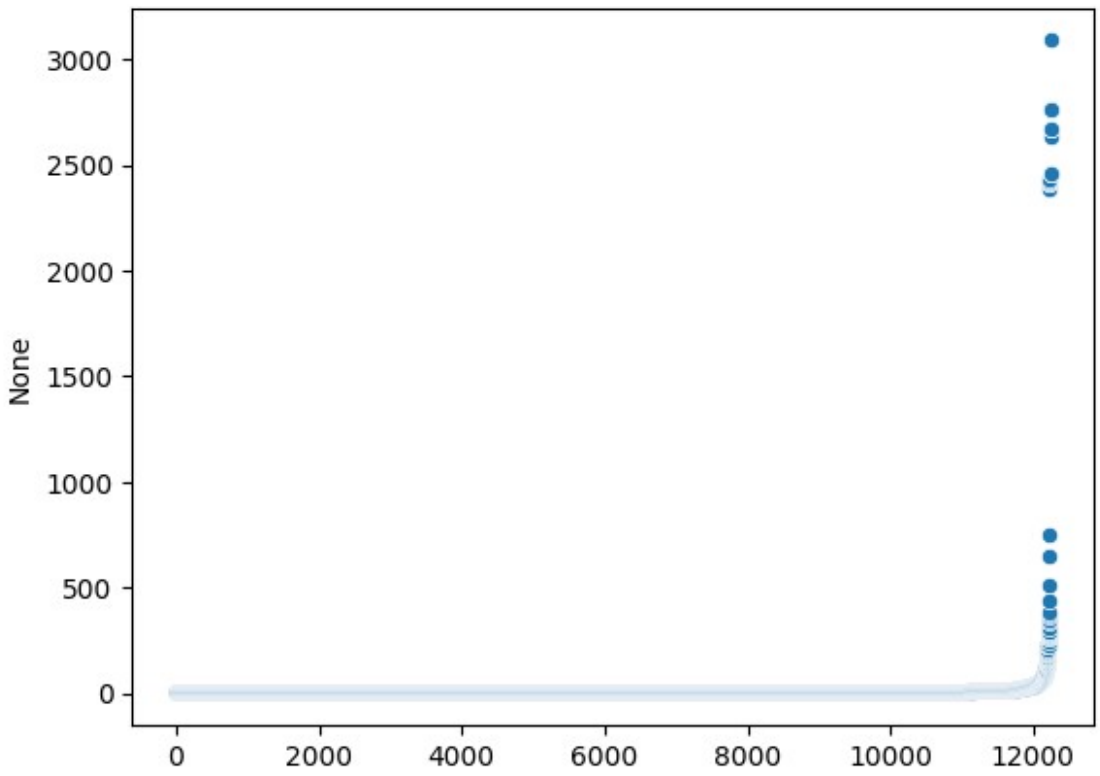


FIGURA 2. Número de características por ítem (Libro)

Cada punto en la figura representa un ítem (Libro) y el número de características que tiene asignadas.

La figura representa el número de características asignadas a cada ítem. Se observa que la mayoría de los ítems tiene una cantidad intermedia de características, mientras que solo unos pocos presentan valores muy bajos o muy altos. Esto indica que los ítems del conjunto de datos no están descritos de manera homogénea: algunos tienen poca información asociada y otros cuentan con una descripción más completa.

- No todos los libros tienen el mismo nivel de detalle
- La mayoría tiene una cantidad razonable de atributos
- Pero hay heterogeneidad: algunos están subrepresentados y otros sobrerrepresentados

De cara al sistema de recomendación:

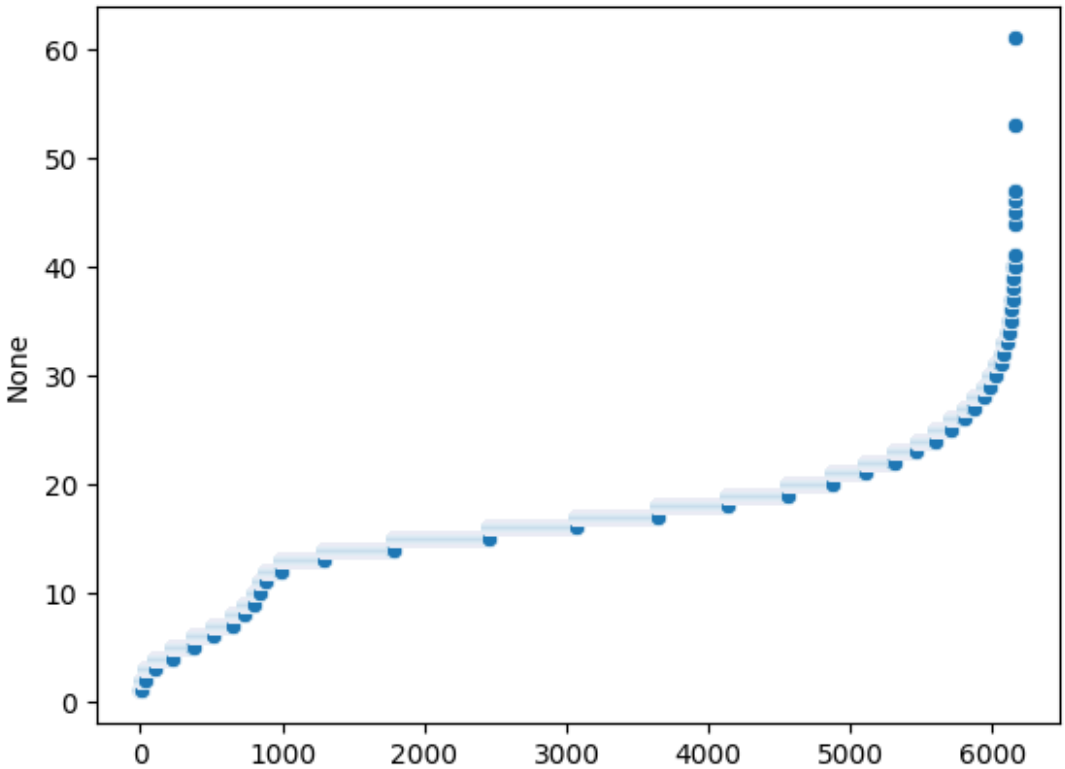
- Los libros con pocas características serán más difíciles de comparar
- Los libros con muchas características pueden generar comparaciones más precisas

```
series_suma_2=df_matriz_libros_concepto.sum(axis=1)
series_suma_2

DBbook_ItemID
1       23
2       20
```

```
3        36
5        23
7        22
         ..
8164     23
8166     13
8167     17
8168     22
8169     25
Length: 6164, dtype: int64
```

```
sns.scatterplot(x=range(0, len(series_suma_2)),
y=series_suma_2.sort_values())
```

```
<Axes: ylabel='None'>
```



Cálculo de matriz tf-idf

La matriz df_matriz_libros_concepto hasta el momento tiene en cada coordenada la presencia o ausencia de la característica que describe el atributo, este sería el término tf de la siguiente fórmula donde i es un término o palabra y d es un documento.

$$\mathrm{tfidf}_{i,d} = \mathrm{tf}_{i,d} \cdot \mathrm{idf}_i$$

El Inverse Document Frequency esta definido como:

$$\mathrm{idf}_{i,d} = \log \frac{N}{\mathrm{df}_i}$$

Donde df_t es el número de documentos en los que aparece el término i y N el número total de documentos

En python es posible operar vectores con escalares directamente, gracias a que por debajo python genera operaciones entre arreglos cuando operamos con un escalar mediante la operación de broadcasting Por ejemplo, el anterior arreglo se puede dividir entre 6, por debajo python genera un arreglo del mismo tamaño y realiza la operación elemento a elemento

```
series_suma/6
```

```
featureID
1          1.000000
2         37.500000
3        461.000000
4        445.333333
5        445.666667
            ...
13997      0.166667
13998      0.166667
13999      0.166667
14000      0.166667
14001      0.166667
Length: 12246, dtype: float64
```

En las siguientes celdas, cree un objeto tipo series llamado df_idf, que contiene el IDF de cada atributo. Puede utilizar la función np.log2 de numpy

Explicación

- len(df_matriz_libros_concepto) → total de libros N
- series_suma → cuántos libros tienen cada atributo dfi
- len(df_matriz_libros_concepto) / series_suma → N/dfi
- np.log2(...) → aplica el logaritmo base 2 a cada elemento

Como series_suma es una Series, el resultado df_idf también será una Series, indexada por featureID.

```
df_idf=np.log2(len(df_matriz_libros_concepto) / series_suma)
df_idf
```

```
featureID
1        10.004689
2         4.775870
3         1.156066
4         1.205947
```

```
5         1.204867
           ...
13997    12.589651
13998    12.589651
13999    12.589651
14000    12.589651
14001    12.589651
Length: 12246, dtype: float64
```

Si df_idf fue calculado correctamente, la siguiente instrucción multiplicará cada fila del dataframe elemento por elemento (element-wise) por la serie que contiene el IDF

```
df_matriz_tf_idf=df_matriz_libros_concepto.multiply(df_idf, axis=1)
df_matriz_tf_idf
```

```
featureID          1        2         3         4         5         6
\
DBbook_ItemID

1              10.004689  4.77587  1.156066  1.205947  1.204867
1.158676
2              10.004689  4.77587  1.156066  1.205947  1.204867
1.158676
3               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
5               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
7               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
...                  ...      ...       ...       ...       ...
...
8164            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
8166            0.000000  0.00000  0.000000  0.000000  0.000000
0.000000
8167            0.000000  0.00000  0.000000  0.000000  0.000000
0.000000
8168            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
8169            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676

featureID         7         8         9         10     ...  13992
13993  \
DBbook_ItemID                                           ...

1              1.158676  6.808291  7.035062  9.589651  ...    0.0
0.0
2              1.158676  0.000000  7.035062  9.589651  ...    0.0
```

```
0.0
3              1.158676  6.808291  0.000000  0.000000  ...    0.0
0.0
5              1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
7              1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
...                 ...       ...       ...       ...  ...    ...    .
..
8164           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
8166           0.000000  0.000000  0.000000  0.000000  ...    0.0
0.0
8167           0.000000  0.000000  0.000000  0.000000  ...    0.0
0.0
8168           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
8169           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0

featureID      13994      13995      13996  13997  13998  13999  14000
14001
DBbook_ItemID

1                0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
2                0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
3                0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
5                0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
7                0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
...              ...        ...        ...    ...    ...    ...    ...
...
8164             0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
8166             0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
8167             0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
8168             0.0   0.000000   0.000000    0.0    0.0    0.0    0.0
0.0
8169             0.0  12.589651  12.589651    0.0    0.0    0.0    0.0
0.0

[6164 rows x 12246 columns]
```

En el siguiente mapa de calor se observa el tf_idf de los items representados en las 300 características con mayor frecuencia.

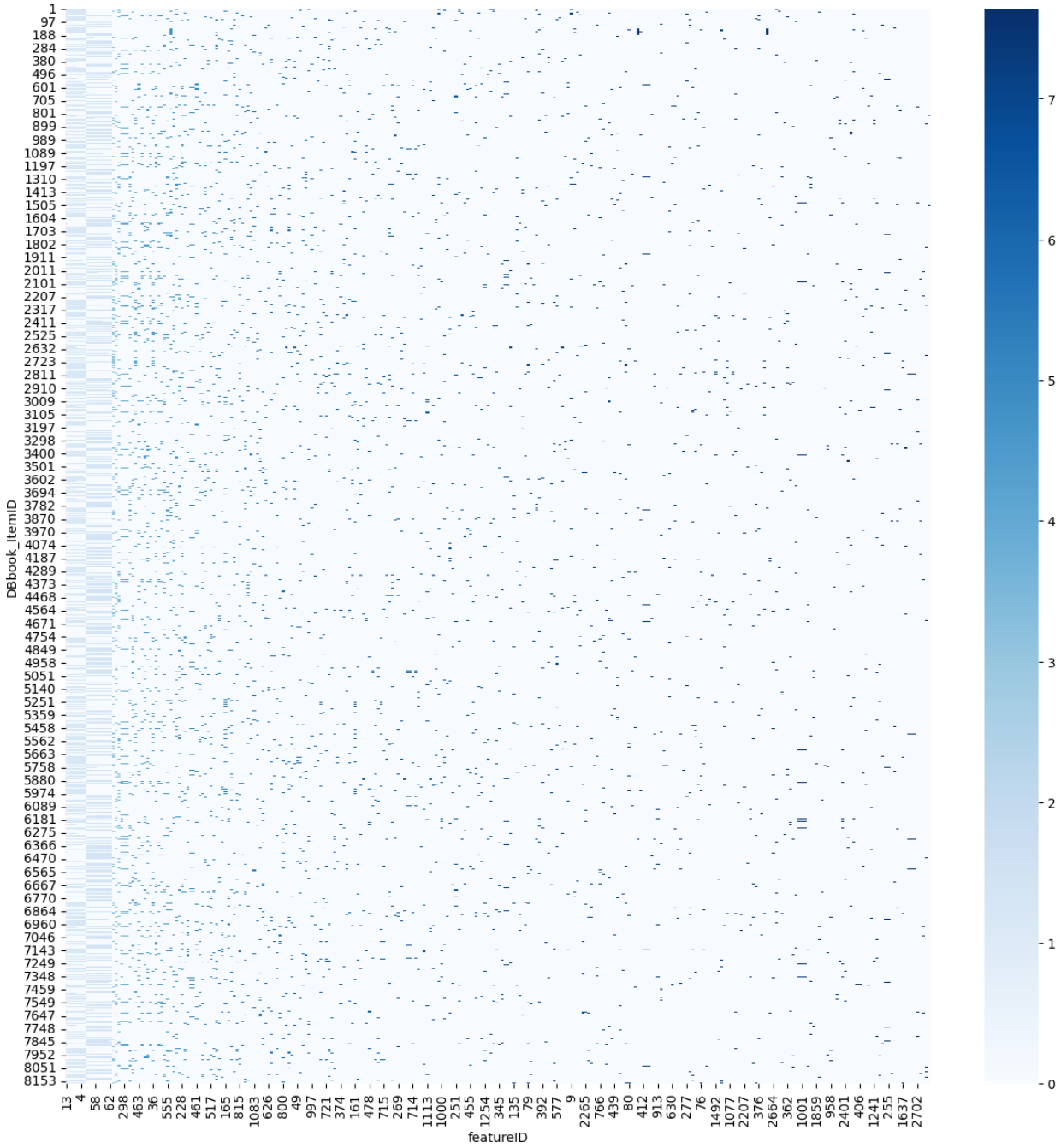
```
df_matriz_tf_idf.loc[:,series_suma.nlargest(300).index]
```

```
featureID          13        3         6         7         5         4
\
DBbook_ItemID

1               0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
2               0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
3               0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
5               0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
7               0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
...                  ...       ...       ...       ...       ...
...
8164            0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
8166            0.000000  0.000000  0.000000  0.000000  0.000000
0.000000
8167            0.000000  0.000000  0.000000  0.000000  0.000000
0.000000
8168            0.996727  1.156066  1.158676  1.158676  1.204867
1.205947
8169            0.996727  1.156066  1.158676  1.158676  1.204867
1.205947

featureID          11        65        59        60    ...      1637
2214  \
DBbook_ItemID                                           ...

1               1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
2               1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
3               1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
5               1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
7               1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
...                  ...       ...       ...       ...  ...       ...
...
8164            1.227159  0.000000  0.000000  0.000000  ...  7.589651
0.0
```

```
8166           0.000000  1.325209  1.325795  1.325795  ...  0.000000
0.0
8167           0.000000  1.325209  1.325795  1.325795  ...  0.000000
0.0
8168           1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0
8169           1.227159  0.000000  0.000000  0.000000  ...  0.000000
0.0

featureID      2227  2228  2230  2702  447   496   1299  1507
DBbook_ItemID
1               0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
2               0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
3               0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
5               0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
7               0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
...              ...   ...   ...   ...   ...   ...   ...   ...
8164            0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
8166            0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
8167            0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
8168            0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0
8169            0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0

[6164 rows x 300 columns]
```

```
plt.rcParams["figure.figsize"] = (15,15)
sns.heatmap(df_matriz_tf_idf.loc[:,series_suma.nlargest(300).index],cm
ap="Blues", vmin=0)
```

```
<Axes: xlabel='featureID', ylabel='DBbook_ItemID'>
```



¿Qué puede interpretar sobre la figura anterior? Encuentre los nombres de las características más frecuentes

El mapa de calor muestra que, aun considerando las 300 características más frecuentes, la matriz TF-IDF sigue siendo dispersa, ya que la mayoría de los valores son cero. Esto indica que cada ítem solo posee una fracción pequeña de las características disponibles. Además, como se trata de atributos frecuentes, sus pesos TF-IDF suelen ser relativamente bajos, por lo que los tonos claros aparecen solo en algunas posiciones. En general, la figura sugiere que los ítems comparten algunas características comunes, pero mantienen descripciones diferenciadas.

Los nombres de las características más frecuentes se obtienen cruzando los featureID con mayor frecuencia en series_suma con el dataframe df_conceptos, que contiene la correspondencia entre featureID y featureShortname.

```
top_10_features = (
    df_conceptos
    .merge(series_suma.rename('frecuencia'),
           left_on='featureID',
           right_index=True)
    .sort_values('frecuencia', ascending=False)
    .head(10)
)

top_10_features
```

```
    featureID                 featureShortname  frecuencia
14         13            /Abstraction100002137        3089
8           3          /Communication100033020        2766
12          7   /WrittenCommunication106349220        2761
11          6                /Writing106362953        2761
10          5    /LiteraryComposition106364329        2674
9           4                /Fiction106367107        2672
13         11                  /Novel106367879        2633
55         65         /PhysicalEntity100001930        2460
50         59                 /Object100002684        2459
51         60                  /Whole100003553        2459
```

Selección de características

Una vez realizado el proceso de indexamiento, se puede realizar el proceso de selección de características.

En este momento contamos con más de 12 mil conceptos. ¿Con cuántos vale la pena crear los modelos de filtrado?

El paso más simple es filtrar las características con baja frecuencia dentro del dataset

```
series_suma.describe()
```

```
count    12246.000000
mean         8.394823
std         95.183540
min          1.000000
25%          1.000000
50%          1.000000
75%          3.000000
max       3089.000000
dtype: float64
```

¿Qué puede decir sobre la distribución de frecuencia de las características en los items?

La mayoría de las características aparece en muy pocos ítems. De hecho, la mediana igual a 1 indica que al menos la mitad de las características están asociadas a un solo ítem.

El 75% de las características aparece en 3 o menos ítems. Solo unas pocas características son muy frecuentes, y por eso el máximo llega hasta 3089. La desviación estándar es muy alta respecto a la media, lo que confirma que hay mucha dispersión y valores extremos.

¿Vale la pena tener todas las características que tenemos actualmente?

No, Muchas de esas características:

- Aparecen en solo 1 o 2 ítems
- Aportan poca capacidad de generalización
- Hacen la matriz más grande y más dispersa
- Pueden introducir ruido en el modelo

Para filtrado basado en contenido, conservar todas puede no ser conveniente porque:

- Aumenta el costo computacional
- Dificulta encontrar similitudes útiles entre ítems
- Muchas características tan raras no ayudan a comparar libros entre sí

Retire de la matriz df_matrix_tf_idf las columnas que representan a los items que tienen menos de 3 items asociados

```
# Se filtra la serie por aquellos que tienen al menos 3
series_suma[series_suma>=3]
```

```
featureID
1           6
2         225
3        2766
4        2672
5        2674
         ...
13204       5
13549       3
13627       3
13920       5
13922       5
Length: 3894, dtype: int64
```

Explicación

- Series_suma[series_suma >= 3] filtra las características que aparecen en al menos 3 ítems

- .index toma los featureID de esas características
- df_matriz_tf_idf.loc[:, ...] selecciona solo esas columnas en la matriz TF-IDF

```
df_matriz_tf_idf = df_matriz_tf_idf.loc[:, series_suma[series_suma >=
3].index]
df_matriz_tf_idf
```

```
featureID          1        2         3         4         5         6
\
DBbook_ItemID

1              10.004689  4.77587  1.156066  1.205947  1.204867
1.158676
2              10.004689  4.77587  1.156066  1.205947  1.204867
1.158676
3               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
5               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
7               0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
...                  ...      ...       ...       ...       ...
...
8164            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
8166            0.000000  0.00000  0.000000  0.000000  0.000000
0.000000
8167            0.000000  0.00000  0.000000  0.000000  0.000000
0.000000
8168            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676
8169            0.000000  0.00000  1.156066  1.205947  1.204867
1.158676

featureID         7         8         9         10     ...  12919
12995  \
DBbook_ItemID                                           ...

1              1.158676  6.808291  7.035062  9.589651  ...    0.0
0.0
2              1.158676  0.000000  7.035062  9.589651  ...    0.0
0.0
3              1.158676  6.808291  0.000000  0.000000  ...    0.0
0.0
5              1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
7              1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
...                 ...       ...       ...       ...  ...    ...    .
..
```

```
8164           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
8166           0.000000  0.000000  0.000000  0.000000  ...    0.0
0.0
8167           0.000000  0.000000  0.000000  0.000000  ...    0.0
0.0
8168           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0
8169           1.158676  0.000000  0.000000  0.000000  ...    0.0
0.0

featureID      12997  13045  13046  13204  13549  13627  13920  13922

DBbook_ItemID

1                0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

2                0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

3                0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

5                0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

7                0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

...              ...    ...    ...    ...    ...    ...    ...    ...

8164             0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

8166             0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

8167             0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

8168             0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0

8169             0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0


[6164 rows x 3894 columns]
```

Para las siguientes estrategias de selección de características, tenemos que aplicar técnicas supervisadas (que conocen la clase a predecir o lo que se quiere pronosticar), para esto tenemos que retomar nuestro dataset original de interacciones entre usuarios e items para asignar la etiqueta (le gustó/ no le gustó)

```
# Recordemos como es el dataset original.
df_dbbook.head(20)
```

```
    DBbook_userID                                name
DBbook_ItemID  \
0               1  Contingency, Irony, and Solidarity            508

1               1  Contingency, Irony, and Solidarity            508

2               1  Contingency, Irony, and Solidarity            508

3               1  Contingency, Irony, and Solidarity            508

4               1  Contingency, Irony, and Solidarity            508

5               1  Contingency, Irony, and Solidarity            508

6               1  Contingency, Irony, and Solidarity            508

7               1  Contingency, Irony, and Solidarity            508

8               1               The Solitaire Mystery            802

9               1               The Solitaire Mystery            802

10              1               The Solitaire Mystery            802

11              1               The Solitaire Mystery            802

12              1               The Solitaire Mystery            802

13              1               The Solitaire Mystery            802

14              1               The Solitaire Mystery            802

15              1               The Solitaire Mystery            802

16              1               The Solitaire Mystery            802

17              1               The Solitaire Mystery            802

18              1               The Solitaire Mystery            802

19              1               The Solitaire Mystery            802


    featureID                              featureShortname  rate
0        1066                        /Category:1989_books     5
1        2300                              /Richard_Rorty     5
2        2301         /Category:Works_about_Marcel_Proust     5
3        2302            /Category:Books_by_Richard_Rorty     5
4        2303                 /Category:Books_about_irony     5
5        2304  /Category:Cambridge_University_Press_books     5
6        2305      /Category:Cognitive_science_literature     5
7        2306                      /Category:Ethics_books     5
```

```
8           3                      /Communication100033020     4
9           4                            /Fiction106367107     4
10          5                /LiteraryComposition106364329     4
11          6                            /Writing106362953     4
12          7               /WrittenCommunication106349220     4
13         11                              /Novel106367879     4
14         13                       /Abstraction100002137     4
15        127                                    /Fantasy     4
16        236                                 /1990Novels     4
17        241                       /Category:1990_novels     4
18        330               /Category:Philosophical_novels     4
19        354                      /Philosophical_fiction     4
```

Creemos un dataframe para crear un dataset de un sistema de recomendación como el visto en el laboratorio pasado (una única interacción de tipo usuario, item y rating)

```
df_all_interactions=df_dbbook[['DBbook_userID','DBbook_ItemID','rate']
].drop_duplicates()
df_all_interactions
```

```
         DBbook_userID  DBbook_ItemID  rate
0                    1            508     5
8                    1            802     4
26                   1            889     5
49                   1           1367     5
72                   1           1736     5
...                ...            ...   ...
1439370           7255           6871     3
1439392           7255           7227     3
1439401           7255           7471     3
1439416           7255           7472     4
1439435           7255           8010     3

[75542 rows x 3 columns]
```

Para asignar una clase, se binarizan los ratings. Una regla simple es calcular el rating promedio por persona. Todo lo que esté por debajo del promedio se clasifica como no le gusta, igual o por encima es si le gusta.

```
df_user_mean=df_all_interactions.groupby('DBbook_userID')
['rate'].mean().reset_index()
df_user_mean.columns=['DBbook_userID','mean']
df_user_mean
```

```
      DBbook_userID      mean
0                 1  4.545455
1                 2  4.250000
2                 3  3.684211
3                 4  3.571429
```

```
4                   5  4.181818
...               ...       ...
6176             7251  3.800000
6177             7252  3.230769
6178             7253  4.000000
6179             7254  4.222222
6180             7255  3.166667

[6181 rows x 2 columns]
```

Realice un merge entre df_all_interactions y df_user_mean, asignando su resultado a df_all_interactions. Cree una nueva columna en el dataframe df_all_interactions llamada 'class' con True si el rating del usuario es mayor o igual a su promedio

Documentación pandas merge

Tutorial creación de columnas a partir del valor de otras

Explicación

- merge(...) agrega a cada interacción el promedio de su usuario
- df_all_interactions['rate'] >= df_all_interactions['mean'] devuelve:
- True si el rating es mayor o igual al promedio del usuario
- False si está por debajo

```
df_all_interactions = df_all_interactions.merge(df_user_mean,
on='DBbook_userID', how='left')

df_all_interactions['class'] = df_all_interactions['rate'] >=
df_all_interactions['mean']

df_all_interactions
```

```
       DBbook_userID  DBbook_ItemID  rate      mean  class
0                  1            508     5  4.545455   True
1                  1            802     4  4.545455  False
2                  1            889     5  4.545455   True
3                  1           1367     5  4.545455   True
4                  1           1736     5  4.545455   True
...              ...            ...   ...       ...    ...
75537           7255           6871     3  3.166667  False
75538           7255           7227     3  3.166667  False
75539           7255           7471     3  3.166667  False
75540           7255           7472     4  3.166667   True
75541           7255           8010     3  3.166667  False

[75542 rows x 5 columns]
```

Por último, se procede a partir el dataset en entrenamiento y test. Se utiliza de la librería sklearn la función train_test_split.

Esta recibe como parámetro el dataset a partir, el porcentaje para test y un parámetro que indica la variable por la cual estratificar la partición, en este caso quisieramos que las interacciones en test sean de usuarios que vimos en train, por lo tanto se deja estratificado por usuario.

```
help(train_test_split)

Help on function train_test_split in module
sklearn.model_selection._split:

train_test_split(*arrays, test_size=None, train_size=None,
random_state=None, shuffle=True, stratify=None)
    Split arrays or matrices into random train and test subsets.

    Quick utility that wraps input validation,
    ``next(ShuffleSplit().split(X, y))``, and application to input
data
    into a single call for splitting (and optionally subsampling) data
into a
    one-liner.

    Read more in the :ref:`User Guide <cross_validation>`.

    Parameters
    ----------
    *arrays : sequence of indexables with same length / shape[0]
        Allowed inputs are lists, numpy arrays, scipy-sparse
        matrices or pandas dataframes.

    test_size : float or int, default=None
        If float, should be between 0.0 and 1.0 and represent the
proportion
        of the dataset to include in the test split. If int,
represents the
        absolute number of test samples. If None, the value is set to
the
        complement of the train size. If ``train_size`` is also None,
it will
        be set to 0.25.

    train_size : float or int, default=None
        If float, should be between 0.0 and 1.0 and represent the
        proportion of the dataset to include in the train split. If
        int, represents the absolute number of train samples. If None,
        the value is automatically set to the complement of the test
size.

    random_state : int, RandomState instance or None, default=None
        Controls the shuffling applied to the data before applying the
split.
        Pass an int for reproducible output across multiple function
```

```
calls.
        See :term:`Glossary <random_state>`.

    shuffle : bool, default=True
        Whether or not to shuffle the data before splitting. If
shuffle=False
        then stratify must be None.

    stratify : array-like, default=None
        If not None, data is split in a stratified fashion, using this
as
        the class labels.
        Read more in the :ref:`User Guide <stratification>`.

    Returns
    -------
    splitting : list, length=2 * len(arrays)
        List containing train-test split of inputs.

        .. versionadded:: 0.16
            If the input is sparse, the output will be a
            ``scipy.sparse.csr_matrix``. Else, output type is the same
as the
            input type.

    Examples
    --------
    >>> import numpy as np
    >>> from sklearn.model_selection import train_test_split
    >>> X, y = np.arange(10).reshape((5, 2)), range(5)
    >>> X
    array([[0, 1],
           [2, 3],
           [4, 5],
           [6, 7],
           [8, 9]])
    >>> list(y)
    [0, 1, 2, 3, 4]

    >>> X_train, X_test, y_train, y_test = train_test_split(
    ...     X, y, test_size=0.33, random_state=42)
    ...
    >>> X_train
    array([[4, 5],
           [0, 1],
           [6, 7]])
    >>> y_train
    [2, 0, 3]
    >>> X_test
```

```
    array([[2, 3],
           [8, 9]])
    >>> y_test
    [1, 4]

    >>> train_test_split(y, shuffle=False)
    [[0, 1, 2], [3, 4]]

    >>> from sklearn import datasets
    >>> iris = datasets.load_iris(as_frame=True)
    >>> X, y = iris['data'], iris['target']
    >>> X.head()
        sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
    0                5.1               3.5                1.4
0.2
    1                4.9               3.0                1.4
0.2
    2                4.7               3.2                1.3
0.2
    3                4.6               3.1                1.5
0.2
    4                5.0               3.6                1.4
0.2
    >>> y.head()
    0    0
    1    0
    2    0
    3    0
    4    0
    ...

    >>> X_train, X_test, y_train, y_test = train_test_split(
    ... X, y, test_size=0.33, random_state=42)
    ...
    >>> X_train.head()
        sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
    96                 5.7               2.9                4.2
1.3
    105                7.6               3.0                6.6
2.1
    66                 5.6               3.0                4.5
1.5
    0                  5.1               3.5                1.4
0.2
    122                7.7               2.8                6.7
2.0
    >>> y_train.head()
```

```
    96     1
    105    2
    66     1
    0      0
    122    2
    ...
    >>> X_test.head()
        sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
    73                 6.1               2.8                4.7
1.2
    18                 5.7               3.8                1.7
0.3
    118                7.7               2.6                6.9
2.3
    78                 6.0               2.9                4.5
1.5
    76                 6.8               2.8                4.8
1.4
    >>> y_test.head()
    73     1
    18     0
    118    2
    78     1
    76     1
    ...
```

```
#Para garantizar reproducibilidad en resultados
seed = 10
random.seed(seed)
np.random.seed(seed)
df_all_interactions_train, df_all_interactions_test
=train_test_split(df_all_interactions, test_size=0.2,
stratify=df_all_interactions['DBbook_userID'])

df_all_interactions_train
```

```
       DBbook_userID  DBbook_ItemID  rate      mean  class
8703             840           2815     4  4.111111  False
64432           6179           4425     3  2.785714   True
73313           7034           6978     5  4.187500   True
22020           2105           2742     4  3.529412   True
31264           2998           7171     3  3.550000  False
...              ...            ...   ...       ...    ...
31712           3035           7494     5  4.250000   True
31592           3026           2094     3  3.285714  False
65130           6246           6288     4  4.200000  False
5437             518           4434     5  4.250000   True
60422           5798           4840     4  4.217391  False
```

```
[60433 rows x 5 columns]
```

```
df_all_interactions_test
```

```
       DBbook_userID  DBbook_ItemID  rate      mean  class
29332           2817           1971     3  3.600000  False
65989           6333           4048     3  3.125000  False
66883           6417           6988     3  4.111111  False
28307           2708           4854     5  4.333333   True
45446           4370           7538     3  3.666667  False
...              ...            ...   ...       ...    ...
53549           5145           1249     3  4.230769  False
63091           6058           2957     5  4.250000   True
34956           3341           1934     3  3.200000  False
73467           7048           3812     3  4.750000  False
64264           6166           2953     4  3.300000   True
```

```
[15109 rows x 5 columns]
```

```
df_all_interactions_train.DBbook_userID.value_counts()
```

```
DBbook_userID
3852    19
6168    19
4908    19
1541    19
3322    19
        ..
7204     4
7165     4
6154     4
5549     4
5463     4
Name: count, Length: 6181, dtype: int64
```

```
df_all_interactions_test.DBbook_userID.value_counts()
```

```
DBbook_userID
6333    5
5307    5
1483    5
158     5
1609    5
       ..
5997    1
5381    1
6973    1
6576    1
3341    1
Name: count, Length: 6181, dtype: int64
```

```
df_conteos_usuario_train_test=pd.concat([df_all_interactions_train.DBb
ook_userID.value_counts(),df_all_interactions_test.DBbook_userID.value
_counts()],axis=1)
df_conteos_usuario_train_test.columns=['train_count','test_count']

df_conteos_usuario_train_test
```

```
               train_count  test_count
DBbook_userID
3852                    19           5
6168                    19           5
4908                    19           5
1541                    19           5
3322                    19           5
...                    ...         ...
7204                     4           1
7165                     4           1
6154                     4           1
5549                     4           1
5463                     4           1

[6181 rows x 2 columns]
```

```
df_conteos_usuario_train_test.nlargest(500,'test_count')
```

```
               train_count  test_count
DBbook_userID
3852                    19           5
6168                    19           5
4908                    19           5
1541                    19           5
3322                    19           5
...                    ...         ...
3334                    18           4
6732                    18           4
4957                    18           4
2918                    18           4
853                     18           4

[500 rows x 2 columns]
```

Chi-cuadrado

La selección de características mediante la prueba chi-cuadrado nos dice si la diferencia observada entre las frecuencias de co-ocurrencia de dos variables es significativa. La idea es seleccionar características que más ayuden a discriminar la clase objetivo observando la frecuencia en la que ocurren juntas.

La librería sklearn permite identificar la importancia de cada una de las variables utilizando el método chi2

```
help(chi2)
```

En filtrado por contenido, se arma un modelo por usuario. Por ahora vamos a escoger las características más importantes para el primer usuario del dataset de test

```
#Debería dar 2817
primer_usuario_id=df_all_interactions_test.iloc[0]['DBbook_userID']
primer_usuario_id
```

```
np.int64(2817)
```

Obtengamos los ids de los items con los que ha interactuado y su opinion

```
df_temporal_usuario=df_all_interactions_train.loc[df_all_interactions_
train.DBbook_userID==1124,['DBbook_ItemID','class']]
df_temporal_usuario
```

```
       DBbook_ItemID  class
11947           5736   True
11944           4108  False
11943           4103   True
11938           1391  False
11937           1292  False
11940           1970   True
11946           4799  False
11941           3006   True
11939           1578   True
```

Peguemos a este dataframe la representación vectorial de tf_idf por el id del item, en la matriz es el índice de las filas

```
df_temporal_usuario=df_temporal_usuario.merge(df_matriz_tf_idf,
how='left', left_on='DBbook_ItemID', right_index=True)
df_temporal_usuario
```

```
       DBbook_ItemID  class    1        2         3         4
5  \
11947           5736   True  0.0  0.00000  1.156066  1.205947
1.204867
11944           4108  False  0.0  0.00000  0.000000  0.000000
0.000000
11943           4103   True  0.0  0.00000  0.000000  0.000000
0.000000
11938           1391  False  0.0  0.00000  0.000000  0.000000
0.000000
11937           1292  False  0.0  0.00000  0.000000  0.000000
0.000000
11940           1970   True  0.0  4.77587  1.156066  1.205947
1.204867
```

```
11946           4799  False  0.0  0.00000  0.000000  0.000000
0.000000
11941           3006   True  0.0  0.00000  0.000000  0.000000
0.000000
11939           1578   True  0.0  0.00000  0.000000  0.000000
0.000000

               6         7    8  ...  12919  12995  12997  13045  13046
13204  \
11947  1.158676  1.158676  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11944  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11943  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11938  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11937  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11940  1.158676  1.158676  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11946  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11941  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0
11939  0.000000  0.000000  0.0  ...    0.0    0.0    0.0    0.0    0.0
0.0

       13549  13627  13920  13922
11947    0.0    0.0    0.0    0.0
11944    0.0    0.0    0.0    0.0
11943    0.0    0.0    0.0    0.0
11938    0.0    0.0    0.0    0.0
11937    0.0    0.0    0.0    0.0
11940    0.0    0.0    0.0    0.0
11946    0.0    0.0    0.0    0.0
11941    0.0    0.0    0.0    0.0
11939    0.0    0.0    0.0    0.0

[9 rows x 3896 columns]
```

Este dataframe representa los datos de entrenamiento del modelo para predicción de una clase binaria (class True es le gusta, class False es no le gusta)

```
#vamos a indexar solamente las columnas que son características, la
prueba chi2 lo compara todas las características contra la clase
objetivo
features=df_matriz_tf_idf.columns
```

```
pesos_chi2, pval=
chi2(df_temporal_usuario[features],df_temporal_usuario['class'])

#La prueba puede arrojar nan
pesos_chi2
```

```
array([       nan, 3.82069596, 1.84970513, ...,        nan,
nan,
              nan], shape=(3894,))
```

```
pval
```

```
array([       nan, 0.05062321, 0.17381769, ...,        nan,
nan,
              nan], shape=(3894,))
```

```
# LLenamos con peso 0 los que no se pudieron calcular
pesos_chi2=np.nan_to_num(pesos_chi2)
pesos_chi2
```

```
array([0.        , 3.82069596, 1.84970513, ..., 0.        ,
0.        ,
       0.        ], shape=(3894,))
```

Se crea una máscara de indexación con los valores que son positivos según la prueba

```
pesos_chi2_mask=pesos_chi2>0

pesos_chi2_mask
```

```
array([False,  True,  True, ..., False, False, False], shape=(3894,))
```

```
features[pesos_chi2_mask]
```

```
Index([    2,     3,     4,     5,     6,     7,    11,    13,    22,
30,
          56,    58,    59,    60,    62,    63,    64,    65,    66,
67,
          86,    92,   116,   127,   132,   173,   186,   192,   196,
215,
         241,   242,   290,   292,   294,   295,   296,   297,   298,
372,
         374,   377,   398,   406,   432,   436,   437,   438,   439,
463,
         474,   478,   490,   499,   504,   523,   528,   555,   556,
579,
         582,   725,   778,   797,   801,  1077,  1289,  1301,  1415,
1731,
        1733,  1833,  1887,  1956,  2211,  2213,  2288,  2299,  3252,
3253,
        3753,  4367,  4368,  4602,  4605,  4606,  4723,  4860,  5057,
```

```
6095,
        6096,  6097,  6986,  8656,  8658,  8659, 10004],
      dtype='int64', name='featureID')
```

El siguiente sería el resultado, se recortaron las columnas de pesos del usuario, dejando 119 features.

```
df_temporal_usuario.loc[:,features[pesos_chi2_mask]]

featureID        2         3         4         5         6
7  \
11947      0.00000  1.156066  1.205947  1.204867  1.158676  1.158676

11944      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11943      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11938      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11937      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11940      4.77587  1.156066  1.205947  1.204867  1.158676  1.158676

11946      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11941      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000

11939      0.00000  0.000000  0.000000  0.000000  0.000000  0.000000


featureID        11        13        22        30  ...       4860
5057  \
11947      1.227159  0.996727  0.000000  0.000000  ...   0.000000
0.000000
11944      0.000000  0.000000  0.000000  3.035062  ...   0.000000
0.000000
11943      0.000000  0.000000  0.000000  0.000000  ...   0.000000
0.000000
11938      0.000000  0.000000  0.000000  0.000000  ...   0.000000
0.000000
11937      0.000000  0.000000  0.000000  0.000000  ...   0.000000
0.000000
11940      1.227159  0.996727  7.460368  0.000000  ...   0.000000
9.004689
11946      0.000000  0.000000  0.000000  0.000000  ...   0.000000
0.000000
11941      0.000000  0.996727  0.000000  0.000000  ...   0.000000
0.000000
11939      0.000000  0.000000  0.000000  0.000000  ...  10.004689
0.000000
```

```
featureID       6095       6096       6097       6986      8656
8658  \
11947       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000
11944       0.000000   0.000000   0.000000   0.000000  9.589651
9.782296
11943       0.000000   0.000000   0.000000  10.004689  0.000000
0.000000
11938       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000
11937       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000
11940      11.004689  11.004689  11.004689   0.000000  0.000000
0.000000
11946       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000
11941       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000
11939       0.000000   0.000000   0.000000   0.000000  0.000000
0.000000

featureID      8659      10004
11947      0.000000  11.004689
11944      9.782296   0.000000
11943      0.000000   0.000000
11938      0.000000   0.000000
11937      0.000000   0.000000
11940      0.000000   0.000000
11946      0.000000   0.000000
11941      0.000000   0.000000
11939      0.000000   0.000000

[9 rows x 97 columns]
```

```
del df_temporal_usuario
```

Complete el código de la siguiente celda, el objetivo es crear un diccionario donde la llave es el id del usuario y el valor es un arreglo con los features seleccionados para el usuario

El proceso que implementamos no esta optimizado, por lo que vamos a armar el modelo solamente para los 500 usuarios con más ratings en el dataset de test.

```
diccionario_usuarios_features={}
# 500 usuarios con más ratings en test
unique_users_test=df_conteos_usuario_train_test.nlargest(500,'test_cou
nt').index
i=0
print(unique_users_test.shape)
for user in unique_users_test:
```

```
  if not user in diccionario_usuarios_features:

df_temporal_usuario=df_all_interactions_train.loc[df_all_interactions_
train.DBbook_userID==user,['DBbook_ItemID','class']]

    df_temporal_usuario=df_temporal_usuario.merge(
        df_matriz_tf_idf,
        how='left',
        left_on='DBbook_ItemID',
        right_index=True
    )

    pesos_chi2, pval = chi2(df_temporal_usuario[features],
df_temporal_usuario['class'])
    pesos_chi2=np.nan_to_num(pesos_chi2)
    pesos_chi2_mask=pesos_chi2>0
    features_usuario=features[pesos_chi2_mask]
    diccionario_usuarios_features[user]=features_usuario
    i=i+1
    if i%50==0:
      print(i)
    del df_temporal_usuario
```

```
(500,)
50
100
150
200
250
300
350
400
450
500
```

```
len(diccionario_usuarios_features)
```

```
500
```

```
diccionario_usuarios_features.keys()
```

```
dict_keys([3852, 6168, 4908, 1541, 3322, 209, 3800, 1726, 4926, 1343,
3219, 2643, 2678, 6366, 4619, 6123, 2107, 2041, 6084, 4627, 4425,
1019, 2288, 6880, 3639, 1512, 4542, 2539, 6988, 5338, 734, 6134, 9,
1728, 23, 2159, 3879, 6556, 4984, 4185, 5591, 5569, 4277, 2172, 7215,
1563, 430, 3824, 107, 3539, 3703, 5137, 2812, 509, 774, 2474, 409,
6585, 1690, 3416, 2455, 938, 188, 2089, 1084, 2262, 4469, 2313, 7248,
351, 2689, 2137, 4500, 5573, 3083, 3921, 1247, 5017, 6335, 6715, 1722,
4040, 4616, 511, 4987, 6219, 2185, 5278, 7068, 5727, 4192, 681, 1660,
2877, 3490, 4877, 6167, 1609, 4124, 643, 254, 2852, 5995, 6627, 189,
158, 4701, 3342, 6873, 2682, 2881, 2914, 1214, 5688, 5951, 4764, 248,
```

```
4234, 664, 2345, 872, 2757, 1934, 321, 3255, 4622, 2097, 1206, 3957,
1076, 5564, 6387, 6546, 3602, 1549, 581, 4466, 4628, 921, 1715, 1176,
3484, 6562, 68, 3119, 4562, 1783, 2610, 5698, 4232, 7007, 4242, 1955,
5261, 3348, 689, 6333, 6432, 3799, 2490, 627, 6487, 6694, 1021, 2160,
2230, 5486, 6115, 2258, 4714, 5931, 6828, 118, 2891, 3839, 1114, 676,
5905, 5186, 7230, 5337, 3981, 4120, 1103, 2193, 2958, 2103, 2701,
1306, 6654, 5033, 547, 69, 6141, 2184, 2840, 5542, 1928, 4452, 701,
4184, 6505, 4743, 4891, 4193, 1341, 5167, 2286, 876, 6060, 4653, 6243,
3858, 2433, 3795, 2847, 175, 3055, 768, 6431, 5985, 4215, 6259, 6110,
6655, 2049, 3956, 1858, 2108, 6755, 1589, 2287, 3031, 3886, 740, 6337,
669, 4573, 201, 7031, 3203, 6664, 2017, 4433, 3823, 784, 3544, 5832,
6039, 2004, 6066, 1553, 3469, 1580, 6036, 4721, 661, 2329, 3153, 6511,
5556, 854, 5301, 6895, 5519, 887, 408, 1483, 2429, 2873, 4568, 6902,
5170, 6173, 2988, 239, 2208, 2742, 5929, 6771, 1044, 857, 1537, 2318,
6625, 4106, 4381, 2050, 3755, 144, 5784, 5910, 3000, 5364, 6700, 5457,
651, 1903, 2730, 4900, 4696, 3220, 3629, 7238, 212, 4243, 2992, 196,
4276, 1671, 1301, 5426, 1438, 5189, 2134, 1415, 976, 524, 206, 4368,
1600, 1754, 4208, 3121, 1640, 3684, 6907, 6067, 1423, 7210, 2705,
4089, 362, 3883, 6888, 2147, 3725, 4970, 6017, 1508, 251, 5050, 5993,
1525, 6814, 3383, 1099, 3990, 6009, 765, 6075, 2524, 5307, 2253, 4404,
617, 215, 1345, 647, 1362, 7106, 2069, 1199, 5880, 5798, 3552, 6691,
3182, 6215, 5576, 4934, 5650, 919, 4873, 6460, 6489, 1036, 5614, 6429,
5174, 3395, 6633, 5068, 2224, 3620, 1294, 1956, 7038, 3410, 4119,
6671, 1965, 7022, 4043, 6377, 931, 2200, 1547, 4685, 406, 1001, 3583,
6962, 29, 5600, 3664, 5735, 7012, 401, 5375, 4187, 4430, 6595, 7244,
843, 1570, 1079, 364, 2296, 943, 3155, 4941, 1980, 5703, 2843, 909,
6451, 4369, 1613, 3702, 5264, 2357, 3572, 1422, 4785, 224, 4047, 4626,
2557, 267, 5732, 1673, 814, 1087, 2485, 6396, 2390, 1268, 6887, 444,
3461, 3764, 7188, 4557, 4501, 7109, 5918, 6149, 6276, 4236, 1685,
2489, 2774, 7243, 104, 3320, 3006, 2212, 6220, 2227, 2963, 3258, 4907,
1475, 5400, 6474, 6578, 3636, 2717, 3511, 148, 6782, 2617, 2818, 5416,
2568, 1165, 7159, 7229, 2448, 3060, 5988, 4655, 1572, 4892, 3334,
6732, 4957, 2918, 853])
```

Otro criterio que puede ser usado es Mutual Information

Modelo de recomendación y evaluación

Una vez seleccionadas las características por usuario, se puede usar el dataset de entrenamiento para aprender un modelo de clasificación binaria y probarlo sobre test.

Uno de los modelos que puede ser utilizado es el clasificador por vecinos más cercanos.

Cada usuario tiene unas instancias representadas en un espacio vectorial del tamaño de las características seleccionadas anteriormente. Para una nueva instancia (predicción) se mira cuáles son los k vecinos más cercanos a ese dato nuevo y se predice la clase mayoritaria dentro del grupo de los vecinos. Observe el siguiente ejemplo.

```
%%html
<iframe src="https://es.wikipedia.org/wiki/K_vecinos_m%C3%A1s_pr
%C3%B3ximos#/media/Archivo:KnnClassification.svg" width="1200"
height="600"></iframe>
```

```
<IPython.core.display.HTML object>
```

Tomemos como ejemplo el usuario 3852, armemos su conjunto de entrenamiento. Note que se estan seleccionando solamente los features calculados en el punto anterior

```
usuario_id = 3852
if usuario_id not in diccionario_usuarios_features:
    usuario_id = next(iter(diccionario_usuarios_features))
    print(f"Usuario 3852 no disponible en entrenamiento. Se usa
usuario {usuario_id}.")

features_usuario = diccionario_usuarios_features[usuario_id]
df_temporal_usuario_train = df_all_interactions_train.loc[
    df_all_interactions_train.DBbook_userID == usuario_id,
['DBbook_ItemID', 'class']
 ]
df_temporal_usuario_train = df_temporal_usuario_train.merge(
    df_matriz_tf_idf[features_usuario], how='left',
left_on='DBbook_ItemID', right_index=True
)
df_temporal_usuario_train
```

```
       DBbook_ItemID  class        2         3         4         5
6  \
40213           7275  False  0.00000  0.000000  0.000000  0.000000
0.000000
40203           3537   True  0.00000  1.156066  1.205947  1.204867
1.158676
40201           2953   True  0.00000  0.000000  0.000000  0.000000
0.000000
40206           5683  False  0.00000  1.156066  1.205947  1.204867
1.158676
40208           6709  False  0.00000  0.000000  0.000000  0.000000
0.000000
40209           6842  False  0.00000  0.000000  0.000000  0.000000
0.000000
40205           5483   True  0.00000  0.000000  0.000000  0.000000
0.000000
40197           2836   True  0.00000  1.156066  1.205947  1.204867
1.158676
40195           2069  False  0.00000  0.000000  0.000000  0.000000
0.000000
40194           2007   True  0.00000  0.000000  0.000000  0.000000
0.000000
```

```
40211           7046  False  0.00000  0.000000  0.000000  0.000000
0.000000
40207           5985   True  0.00000  1.156066  1.205947  1.204867
1.158676
40214           7732  False  0.00000  0.000000  0.000000  0.000000
0.000000
40199           2838   True  0.00000  0.000000  0.000000  0.000000
0.000000
40192            111   True  0.00000  0.000000  0.000000  0.000000
0.000000
40198           2837  False  0.00000  0.000000  0.000000  0.000000
0.000000
40191             59  False  0.00000  0.000000  0.000000  0.000000
0.000000
40202           3278   True  0.00000  0.000000  0.000000  0.000000
0.000000
40193            385   True  4.77587  0.000000  0.000000  0.000000
0.000000

               7         11         13  ...      4045      4783
4913  \
40213  0.000000  0.000000  0.000000  ...  0.000000  7.945795
11.004689
40203  1.158676  1.227159  0.996727  ...  0.000000  0.000000
0.000000
40201  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40206  1.158676  1.227159  0.996727  ...  0.000000  0.000000
0.000000
40208  0.000000  0.000000  0.000000  ...  8.782296  0.000000
0.000000
40209  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40205  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40197  1.158676  1.227159  0.996727  ...  0.000000  0.000000
0.000000
40195  0.000000  0.000000  0.996727  ...  0.000000  0.000000
0.000000
40194  0.000000  0.000000  0.996727  ...  0.000000  0.000000
0.000000
40211  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40207  1.158676  1.227159  0.996727  ...  0.000000  0.000000
0.000000
40214  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40199  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
```

```
40192  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40198  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40191  0.000000  0.000000  0.996727  ...  0.000000  0.000000
0.000000
40202  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000
40193  0.000000  0.000000  0.000000  ...  0.000000  0.000000
0.000000

           5017       6186       6187       6318       7650       7651
\
40213  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40203  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40201  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40206  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40208  8.267723   0.000000   0.000000   0.000000   0.000000   0.000000

40209  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40205  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40197  0.000000   0.000000   0.000000   0.000000  10.004689  10.589651

40195  0.000000   0.000000   0.000000  11.004689   0.000000   0.000000

40194  0.000000  10.589651  10.589651   0.000000   0.000000   0.000000

40211  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40207  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40214  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40199  0.000000   0.000000   0.000000   0.000000  10.004689  10.589651

40192  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40198  0.000000   0.000000   0.000000   0.000000  10.004689   0.000000

40191  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000

40202  0.000000   0.000000   0.000000   0.000000  10.004689  10.589651

40193  0.000000   0.000000   0.000000   0.000000   0.000000   0.000000
```

```
            8839
40213   0.000000
40203  11.004689
40201   0.000000
40206   0.000000
40208   0.000000
40209   0.000000
40205   0.000000
40197   0.000000
40195   0.000000
40194   0.000000
40211   0.000000
40207   0.000000
40214   0.000000
40199   0.000000
40192   0.000000
40198   0.000000
40191   0.000000
40202   0.000000
40193   0.000000

[19 rows x 124 columns]
```

Armamos de igual forma el conjunto de test

```
df_temporal_usuario_test = df_all_interactions_test.loc[
    df_all_interactions_test.DBbook_userID == usuario_id,
['DBbook_ItemID', 'class']
 ]
df_temporal_usuario_test = df_temporal_usuario_test.merge(
    df_matriz_tf_idf[features_usuario], how='left',
left_on='DBbook_ItemID', right_index=True
)
df_temporal_usuario_test
```

```
       DBbook_ItemID  class    2    3    4    5    6    7   11
13  ...  \
40196           2578   True  0.0  0.0  0.0  0.0  0.0  0.0  0.0
0.000000  ...
40210           6965  False  0.0  0.0  0.0  0.0  0.0  0.0  0.0
0.996727  ...
40200           2839  False  0.0  0.0  0.0  0.0  0.0  0.0  0.0
0.000000  ...
40212           7271  False  0.0  0.0  0.0  0.0  0.0  0.0  0.0
0.000000  ...
40204           4760  False  0.0  0.0  0.0  0.0  0.0  0.0  0.0
0.000000  ...
```

```
       4045  4783  4913  5017       6186       6187  6318       7650
7651  \
40196   0.0   0.0   0.0   0.0  10.589651  10.589651   0.0   0.000000
0.0
40210   0.0   0.0   0.0   0.0   0.000000   0.000000   0.0   0.000000
0.0
40200   0.0   0.0   0.0   0.0   0.000000   0.000000   0.0  10.004689
0.0
40212   0.0   0.0   0.0   0.0   0.000000   0.000000   0.0   0.000000
0.0
40204   0.0   0.0   0.0   0.0   0.000000   0.000000   0.0   0.000000
0.0

       8839
40196   0.0
40210   0.0
40200   0.0
40212   0.0
40204   0.0

[5 rows x 124 columns]
```

Utilizaremos la clase KNeighborsClassifier de scikit-learn para hacer la predicción de los datos del conjunto de test. El modelo tiene 3 métodos principales. El constructor permite inicializar el k a usar, la métrica entre otros; fit sirve para darle los datos de entrenamiento base al modelo; y predict para predecir los datos que se le pasan.

```
#Con esta configuración se utilizan los 3 vecinos más cercanos, con
distancia euclidiana
knn_clasif=KNeighborsClassifier(3)

# Fit recibe la matriz de entrenamiento y la clase objetivo
knn_clasif.fit(df_temporal_usuario_train[features_usuario],
df_temporal_usuario_train['class'])
```

```
KNeighborsClassifier(n_neighbors=3)
```

```
# llamamos predict sobre  los test , creando una nueva columna en el
dataframe de test
df_temporal_usuario_test['predict']=knn_clasif.predict(df_temporal_usu
ario_test[features_usuario])

df_temporal_usuario_test[['DBbook_ItemID','class','predict']].merge(df
_libros, how='left', on='DBbook_ItemID')
```

```
   DBbook_ItemID  class  predict                        name
0           2578   True     True         A Room with a View
1           6965  False    False        Desert of the Heart
2           2839  False     True       The Bone Doll's Twin
```

```
3           7271  False    False  Mr. Midshipman Hornblower
4           4760  False    False             Geography Club
```

En las siguientes celdas realice hipótesis sobre por qué falló la clasificación para estos items y por qué funcionó para los otros, revise los conceptos seleccionados para el usuario y los asociados a los items

El libro cae en una zona del espacio vectorial donde predominan vecinos positivos, aunque el caso más cercano sea negativo.

Entre las features activas del libro, las de mayor peso son:

- /Lynn_Flewelling → 10.00
- /Category:HarperCollins_books → 4.78
- /Category:American_fantasy_novels → 4.09
- /Fantasy_literature → 3.60

Eso sugiere que el modelo está dándole mucho peso a señales muy específicas como:

- La autora
- El género fantasy
- Categorías editoriales

El error de clasificación en The Bone Doll's Twin parece deberse a que el modelo KNN tomó una decisión por mayoría entre vecinos cercanos que comparten características de fantasía y posiblemente de autor. Aunque el vecino más próximo fue Hidden Warrior, con clase negativa, los otros dos vecinos más cercanos (Traitor's Moon y Stalking Darkness) pertenecían a la clase positiva, por lo que la predicción final fue True. Esto sugiere que atributos como /Lynn_Flewelling, /Fantasy_literature y /Category:American_fantasy_novels tienen un peso importante en la similitud, pero no necesariamente capturan por completo la preferencia real del usuario sobre ese título particular.

Para sustentar bien la hipótesis, conviene responder las siguientes preguntas mediante código:

Resumen de evidencia por feature para el usuario

- ¿Al usuario le gustan libros con Fantasy_literature?
- ¿Le gustan libros de Lynn_Flewelling?
- ¿Qué tan frecuentes son esas features en positivos vs negativos?

```
# features clave del item fallido
features_clave = [92, 556, 141, 7650]   # Fantasy,
American_fantasy_novels, HarperCollins, Lynn_Flewelling

# interacciones de train del usuario
items_usuario = df_all_interactions_train.loc[
    df_all_interactions_train['DBbook_userID'] == usuario_id,
    ['DBbook_ItemID', 'class']
```

```
]

# conceptos de esos items
conceptos_usuario = items_usuario.merge(
    df_libros_concepto[['DBbook_ItemID', 'featureID',
'featureShortname']].drop_duplicates(),
    on='DBbook_ItemID',
    how='left'
)

# resumen por feature
resumen = (

conceptos_usuario[conceptos_usuario['featureID'].isin(features_clave)]
    .groupby(['featureID', 'featureShortname', 'class'])
    .size()
    .unstack(fill_value=0)
    .reset_index()
    .rename(columns={True: 'n_true', False: 'n_false'})
)

if 'n_true' not in resumen.columns:
    resumen['n_true'] = 0
if 'n_false' not in resumen.columns:
    resumen['n_false'] = 0

resumen['total'] = resumen['n_true'] + resumen['n_false']
resumen['proporcion_true'] = resumen['n_true'] / resumen['total']

resumen.sort_values('proporcion_true', ascending=False)
```

```
class  featureID                   featureShortname  n_false  n_true
total  \
0             92                /Fantasy_literature        1       3
4
3           7650                   /Lynn_Flewelling        1       3
4
2            556  /Category:American_fantasy_novels        2       3
5
1            141      /Category:HarperCollins_books        2       0
2

class  proporcion_true
0                 0.75
3                 0.75
2                 0.60
1                 0.00
```

Qué libros del usuario sustentan esa evidencia

- ¿Cuáles son los libros del usuario donde aparece cada feature?

- ¿Esos libros fueron True o False?

```
evidencia_items = (
    items_usuario
    .merge(

df_libros_concepto[df_libros_concepto['featureID'].isin(features_clave
)]
        [['DBbook_ItemID', 'featureID',
'featureShortname']].drop_duplicates(),
        on='DBbook_ItemID',
        how='inner'
    )
    .merge(
        df_libros[['DBbook_ItemID', 'name']],
        on='DBbook_ItemID',
        how='left'
    )
    .sort_values(['featureID', 'class'], ascending=[True, False])
)

evidencia_items
```

```
    DBbook_ItemID  class  featureID                    featureShortname
\
2            2836   True         92                /Fantasy_literature

5            2838   True         92                /Fantasy_literature

12           3278   True         92                /Fantasy_literature

8            2837  False         92                /Fantasy_literature

1            6709  False        141      /Category:HarperCollins_books

9            2837  False        141      /Category:HarperCollins_books

3            2836   True        556  /Category:American_fantasy_novels

6            2838   True        556  /Category:American_fantasy_novels

13           3278   True        556  /Category:American_fantasy_novels

0            7275  False        556  /Category:American_fantasy_novels

10           2837  False        556  /Category:American_fantasy_novels

4            2836   True       7650                   /Lynn_Flewelling

7            2838   True       7650                   /Lynn_Flewelling

14           3278   True       7650                   /Lynn_Flewelling
```

```
11           2837  False       7650                    /Lynn_Flewelling


                    name
2   Luck in the Shadows
5        Traitor's Moon
12    Stalking Darkness
8        Hidden Warrior
1      The Reptile Room
9        Hidden Warrior
3   Luck in the Shadows
6        Traitor's Moon
13    Stalking Darkness
0         The Firebrand
10       Hidden Warrior
4   Luck in the Shadows
7        Traitor's Moon
14    Stalking Darkness
11       Hidden Warrior
```

Comparar ítem fallido vs vecinos

- ¿Los vecinos positivos comparten esas features con el ítem fallido?

```
item_error = 2839
ids_revisar = [2839, 2837, 2838, 3278]   # item fallido + vecinos

comparacion_vecinos = (
    df_libros_concepto[
        (df_libros_concepto['DBbook_ItemID'].isin(ids_revisar)) &
        (df_libros_concepto['featureID'].isin(features_clave))
    ][['DBbook_ItemID', 'name', 'featureID', 'featureShortname']]
    .drop_duplicates()
    .sort_values(['DBbook_ItemID', 'featureID'])
)

comparacion_vecinos
```

```
        DBbook_ItemID                  name  featureID  \
444282           2837        Hidden Warrior         92
444283           2837        Hidden Warrior        141
444285           2837        Hidden Warrior        556
444286           2837        Hidden Warrior       7650
9270             2838        Traitor's Moon         92
9271             2838        Traitor's Moon        556
9273             2838        Traitor's Moon       7650
9284             2839  The Bone Doll's Twin         92
9286             2839  The Bone Doll's Twin        141
9287             2839  The Bone Doll's Twin        556
9288             2839  The Bone Doll's Twin       7650
```

```
9298             3278     Stalking Darkness         92
9299             3278     Stalking Darkness        556
9301             3278     Stalking Darkness       7650

                          featureShortname
444282                /Fantasy_literature
444283      /Category:HarperCollins_books
444285  /Category:American_fantasy_novels
444286                   /Lynn_Flewelling
9270                  /Fantasy_literature
9271    /Category:American_fantasy_novels
9273                     /Lynn_Flewelling
9284                  /Fantasy_literature
9286        /Category:HarperCollins_books
9287    /Category:American_fantasy_novels
9288                     /Lynn_Flewelling
9298                  /Fantasy_literature
9299    /Category:American_fantasy_novels
9301                     /Lynn_Flewelling
```

Conclusión

La clasificación falló porque The Bone Doll's Twin comparte con sus vecinos varias características que, para este usuario, están más asociadas a ítems positivos que negativos, especialmente /Fantasy_literature, /Lynn_Flewelling y /Category:American_fantasy_novels. Aunque el libro también tiene la característica /Category:HarperCollins_books, que en este usuario se relaciona con ítems negativos, el patrón dominante de similitud quedó sesgado hacia la clase positiva, por lo que el modelo predijo True en lugar de False.

Finalmente, la librería sklearn tiene diferentes métricas de evaluación de clasificación. En particular podemos calcular la matriz de confusión de la clasificación utilizando la función confusion_matrix, y el cálculo de las métricas precision, recall, y f1 con la función precision_recall_fscore_support

```
tn, fp, fn, tp =
confusion_matrix(df_temporal_usuario_test['class'],df_temporal_usuario
_test['predict'], labels=[False,True]).ravel()
```

```
(tn, fp, fn, tp)
```

```
(np.int64(3), np.int64(1), np.int64(0), np.int64(1))
```

```
precision_recall_fscore_support(df_temporal_usuario_test['class'],df_t
emporal_usuario_test['predict'], pos_label=True,average='binary')
```

```
(0.5, 1.0, 0.6666666666666666, None)
```

Ejercicio: Realice las predicciones binarias para los usuarios en el conjunto de test a los que se les hizo la selección de características, mida la precisión, el recall y f_score de su modelo con las predicciones realizadas y ajústelo cambiando el k

1. Evaluar el modelo para todos los usuarios seleccionados
   - Recorre los usuarios del diccionario
   - Arma train y test por usuario
   - Entrena KNN
   - Predice
   - Guarda las predicciones
   - Al final calcula precision, recall y F1 globales

```
def evaluar_knn_usuarios(k, diccionario_usuarios_features,
df_all_interactions_train,
                         df_all_interactions_test, df_matriz_tf_idf):

    predicciones_todas = []

    for user, features_usuario in
diccionario_usuarios_features.items():

        # train del usuario
        df_train_user = df_all_interactions_train.loc[
            df_all_interactions_train['DBbook_userID'] == user,
            ['DBbook_ItemID', 'class']
        ].merge(
            df_matriz_tf_idf[features_usuario],
            how='left',
            left_on='DBbook_ItemID',
            right_index=True
        )

        # test del usuario
        df_test_user = df_all_interactions_test.loc[
            df_all_interactions_test['DBbook_userID'] == user,
            ['DBbook_ItemID', 'class']
        ].merge(
            df_matriz_tf_idf[features_usuario],
            how='left',
            left_on='DBbook_ItemID',
            right_index=True
        )

        # saltar usuarios sin datos suficientes
        if df_train_user.empty or df_test_user.empty:
            continue

        # KNN necesita al menos k ejemplos en train
        if len(df_train_user) < k:
            continue

        # si train tiene una sola clase, KNN no aporta mucho; se puede
saltar
        if df_train_user['class'].nunique() < 2:
```

```
            continue

        X_train = df_train_user[features_usuario]
        y_train = df_train_user['class']

        X_test = df_test_user[features_usuario]
        y_test = df_test_user['class']

        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)

        df_pred_user = pd.DataFrame({
            'DBbook_userID': user,
            'DBbook_ItemID': df_test_user['DBbook_ItemID'].values,
            'class_real': y_test.values,
            'class_pred': y_pred
        })

        predicciones_todas.append(df_pred_user)

    if len(predicciones_todas) == 0:
        return None, None

    df_predicciones = pd.concat(predicciones_todas, ignore_index=True)

    precision, recall, f1, support = precision_recall_fscore_support(
        df_predicciones['class_real'],
        df_predicciones['class_pred'],
        pos_label=True,
        average='binary'
    )

    metricas = pd.DataFrame([{
        'k': k,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'support': support,
        'n_predicciones': len(df_predicciones),
        'n_usuarios_evaluados':
df_predicciones['DBbook_userID'].nunique()
    }])

    return df_predicciones, metricas

df_predicciones_k3, metricas_k3 = evaluar_knn_usuarios(
    k=3,
    diccionario_usuarios_features=diccionario_usuarios_features,
    df_all_interactions_train=df_all_interactions_train,
```

```
    df_all_interactions_test=df_all_interactions_test,
    df_matriz_tf_idf=df_matriz_tf_idf
)

metricas_k3
```

```
   k  precision    recall        f1 support  n_predicciones  \
0  3   0.587142  0.613765  0.600158    None            2375

   n_usuarios_evaluados
0                   493
```

2. Ajustar el modelo cambiando k

```
resultados_k = []

for k in [1, 3, 5, 7, 9, 11]:
    _, metricas_k = evaluar_knn_usuarios(
        k=k,
        diccionario_usuarios_features=diccionario_usuarios_features,
        df_all_interactions_train=df_all_interactions_train,
        df_all_interactions_test=df_all_interactions_test,
        df_matriz_tf_idf=df_matriz_tf_idf
    )

    if metricas_k is not None:
        resultados_k.append(metricas_k)

df_resultados_k = pd.concat(resultados_k, ignore_index=True)
df_resultados_k.sort_values('f1', ascending=False)
```

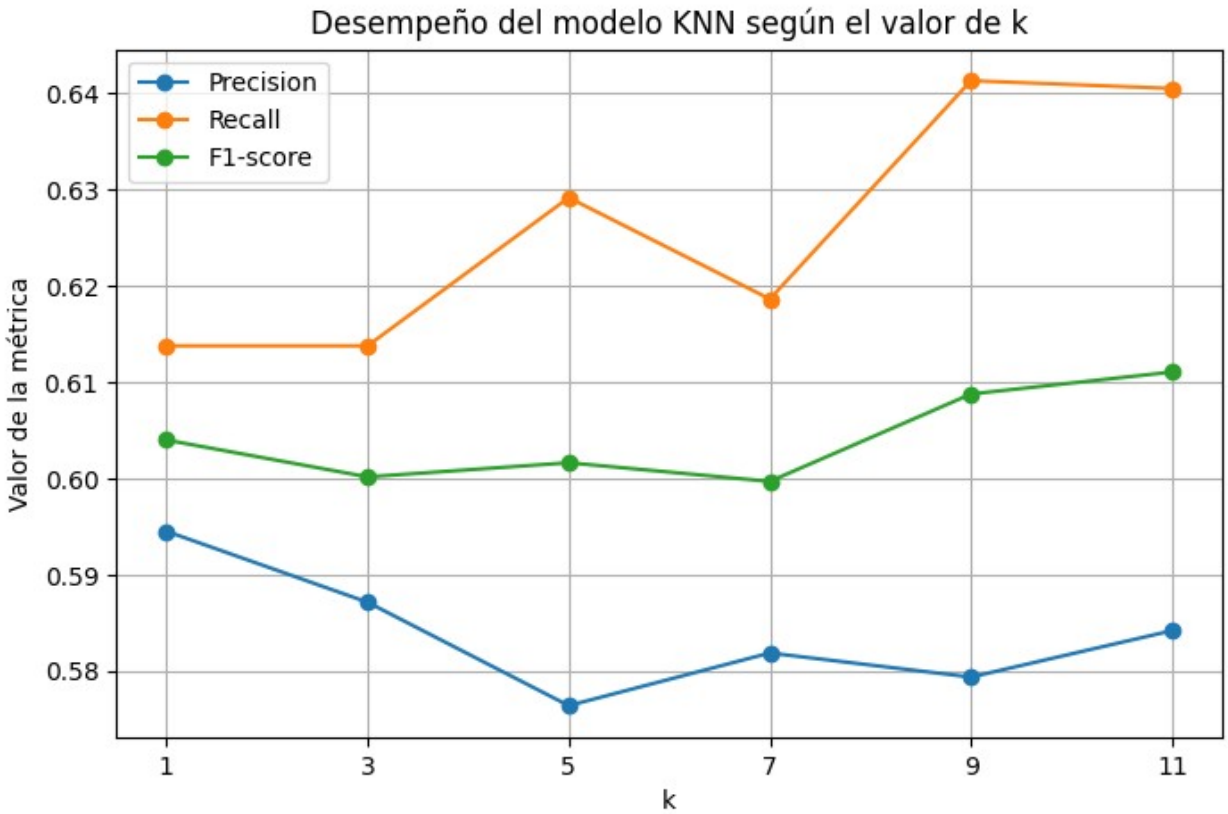
```
    k  precision    recall        f1 support  n_predicciones  \
5  11   0.584195  0.640486  0.611047    None            2375
4   9   0.579371  0.641296  0.608762    None            2375
0   1   0.594510  0.613765  0.603984    None            2375
2   5   0.576409  0.629150  0.601626    None            2375
1   3   0.587142  0.613765  0.600158    None            2375
3   7   0.581874  0.618623  0.599686    None            2375

   n_usuarios_evaluados
5                   493
4                   493
0                   493
2                   493
1                   493
3                   493
```

Mejor K

```
mejor_k = df_resultados_k.sort_values('f1', ascending=False).iloc[0]
mejor_k
```

```
k                                11
precision                  0.584195
recall                     0.640486
f1                         0.611047
support                        None
n_predicciones                 2375
n_usuarios_evaluados            493
Name: 5, dtype: object
```

```
df_resultados_k = pd.DataFrame({
    'k': [1, 3, 5, 7, 9, 11],
    'precision': [0.594510, 0.587142, 0.576409, 0.581874, 0.579371,
0.584195],
    'recall': [0.613765, 0.613765, 0.629150, 0.618623, 0.641296,
0.640486],
    'f1': [0.603984, 0.600158, 0.601626, 0.599686, 0.608762, 0.611047]
})

plt.figure(figsize=(8,5))
plt.plot(df_resultados_k['k'], df_resultados_k['precision'],
marker='o', label='Precision')
plt.plot(df_resultados_k['k'], df_resultados_k['recall'], marker='o',
label='Recall')
plt.plot(df_resultados_k['k'], df_resultados_k['f1'], marker='o',
label='F1-score')

plt.xlabel('k')
plt.ylabel('Valor de la métrica')
plt.title('Desempeño del modelo KNN según el valor de k')
plt.xticks(df_resultados_k['k'])
plt.legend()
plt.grid(True)
plt.show()
```



Conclusión

La gráfica muestra que al variar el número de vecinos k, el desempeño del modelo cambia ligeramente. Aunque $k = 1$ obtiene la mayor precisión, los valores más altos de k mejoran el recall. El mejor equilibrio entre ambas métricas se alcanza con k=11, que obtiene el mayor F1-score, por lo que se selecciona como el valor óptimo para este experimento.

Parte 2: De TF-IDF a Embeddings Densos — Introducción a Word2Vec

Hasta aquí hemos trabajado con representaciones **dispersas** (sparse) basadas en TF-IDF. Estas tienen algunas limitaciones:

- **Alta dimensionalidad:** la matriz tiene miles de columnas (una por concepto)
- **Dispersión:** la mayoría de los valores son cero
- **Sin semántica:** dos conceptos sinónimos tienen columnas diferentes y no hay relación entre ellos

Las representaciones **densas** (como Word2Vec) resuelven estos problemas al aprender vectores de baja dimensionalidad donde conceptos similares están cerca en el espacio vectorial.

A continuación exploraremos paso a paso cómo funciona Word2Vec.

Ejercicio de reflexión: TF-IDF vs Embeddings Densos

Antes de continuar con Word2Vec, reflexione sobre las siguientes preguntas:

1. ¿Qué ventajas y desventajas tiene la representación TF-IDF que utilizamos para los libros?
2. ¿Qué pasaría si dos libros tratan del mismo tema pero usan conceptos diferentes en DBpedia?
3. ¿Cómo cree que un embedding denso podría mejorar las recomendaciones?

Respuesta 1:

La representación TF-IDF tiene como ventaja que es simple de construir, fácil de interpretar y permite identificar qué conceptos son más importantes para describir cada libro. Además, reduce el peso de los conceptos muy frecuentes y resalta los más específicos. Sin embargo, tiene varias desventajas: genera una matriz de muy alta dimensionalidad, es dispersa porque la mayoría de valores son cero, y no captura relaciones semánticas entre conceptos. Dos conceptos parecidos o sinónimos se tratan como columnas totalmente distintas.

Respuesta 2:

Si dos libros tratan del mismo tema pero usan conceptos diferentes en DBpedia, con TF-IDF pueden verse como poco similares o incluso como no relacionados. Esto ocurre porque la comparación depende de coincidencias exactas entre conceptos. Entonces, aunque dos libros sean semánticamente cercanos, si no comparten los mismos términos o features, el modelo puede no detectar esa similitud correctamente.

Respuesta 3:

Un embedding denso podría mejorar las recomendaciones porque representa cada concepto o cada libro en un espacio vectorial de baja dimensión donde conceptos similares quedan cerca entre sí. Así, aunque dos libros no compartan exactamente los mismos conceptos, el modelo podría reconocer que hablan de temas relacionados. Esto permitiría capturar mejor la semántica, reducir la dispersión y generar recomendaciones más robustas.

1. Configuración Inicial

Primero importamos las librerías necesarias y configuramos el logging para ver el progreso del entrenamiento.

```
%matplotlib inline

import logging
logging.basicConfig(format='%(asctime)s : %(levelname)s : %
(message)s', level=logging.INFO)
```

2. ¿Por Qué Necesitamos Representar Palabras como Vectores?

Las computadoras no entienden texto directamente — necesitan números. La pregunta clave es: **¿cómo convertimos palabras en números de forma que se preserve su significado?**

2.1 El Modelo Bolsa de Palabras (Bag-of-Words)

El enfoque más simple es el modelo **Bolsa de Palabras** (*Bag-of-Words*). Este transforma cada documento en un vector de longitud fija donde cada elemento cuenta cuántas veces aparece una palabra.

Ejemplo: Dadas las oraciones:

- *"Juan quiere ver películas. María quiere películas también."*
- *"Juan también quiere ver fútbol. María odia el fútbol."*

El modelo genera vectores como:

- `[1, 2, 1, 1, 2, 1, 1, 0, 0, 0, 0]`
- `[1, 1, 1, 1, 0, 1, 0, 1, 2, 1, 1]`

Donde cada posición corresponde a una palabra del vocabulario.

2.2 Limitaciones de la Bolsa de Palabras

Este enfoque tiene **dos problemas fundamentales**:

1. **Pierde el orden de las palabras:** "Juan quiere a María" y "María quiere a Juan" producen vectores idénticos, aunque significan cosas distintas.

2. **No captura el significado:** Las palabras "bueno" y "excelente" podrían estar tan lejos en el espacio vectorial como "bueno" y "zapato", a pesar de que las dos primeras son sinónimos.

Word2Vec resuelve el segundo problema: aprende representaciones donde palabras con significados similares están cerca en el espacio vectorial.

3. ¿Cómo Funciona Word2Vec?

Word2Vec es un algoritmo basado en redes neuronales superficiales (*shallow neural networks*) que aprende representaciones vectoriales de palabras a partir de grandes cantidades de texto.

3.1 La Idea Central

> **"Conocerás una palabra por la compañía que mantiene"** — J.R. Firth, 1957

Word2Vec se basa en la **hipótesis distribucional**: palabras que aparecen en contextos similares tienden a tener significados similares. Por ejemplo, "perro" y "gato" aparecen frecuentemente con palabras como "mascota", "veterinario", "comida", etc.

3.2 Las Dos Arquitecturas

Word2Vec tiene dos variantes:

Skip-gram (SG)

- **Entrada:** una palabra central
- **Salida:** predice las palabras del contexto (vecinas)
- Ejemplo: dada la palabra "gato", predice "el", "come", "pescado"
- Funciona mejor con corpus pequeños y palabras poco frecuentes

Continuous Bag-of-Words (CBOW)

- **Entrada:** las palabras del contexto
- **Salida:** predice la palabra central
- Ejemplo: dadas "el", "come", "pescado", predice "gato"
- Es más rápido de entrenar y funciona bien con palabras frecuentes

3.3 ¿Qué aprende la red?

La red neuronal tiene una capa oculta. Los **pesos de la capa de proyección** (entre la entrada y la capa oculta) son los **vectores de las palabras** (embeddings). Si la capa oculta tiene 300 neuronas, obtendremos embeddings de 300 dimensiones.

El resultado son vectores con propiedades algebraicas notables, como:

- `vec("rey") - vec("hombre") + vec("mujer") ≈ vec("reina")`
- `vec("París") - vec("Francia") + vec("Japón") ≈ vec("Tokio")`

4. Demostración con un Modelo Pre-entrenado

Antes de entrenar nuestro propio modelo, veamos qué puede hacer Word2Vec usando un modelo ya entrenado con parte del dataset de Google News (~3 millones de palabras y frases).

> **Nota:** El modelo pesa aproximadamente 2GB. Si no tienes buena conexión, puedes saltar a la sección 5 (Entrenar tu propio modelo).

```
%pip install gensim
%pip install plotly
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp311-cp311-macosx_11_0_arm64.whl.metadata
(8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in
./.venv/lib/python3.11/site-packages (from gensim) (2.4.2)
Requirement already satisfied: scipy>=1.7.0 in
./.venv/lib/python3.11/site-packages (from gensim) (1.17.1)
Collecting smart_open>=1.8.1 (from gensim)
  Downloading smart_open-7.5.1-py3-none-any.whl.metadata (24 kB)
Collecting wrapt (from smart_open>=1.8.1->gensim)
  Downloading wrapt-2.1.2-cp311-cp311-macosx_11_0_arm64.whl.metadata
```

```
(7.4 kB)
Downloading gensim-4.4.0-cp311-cp311-macosx_11_0_arm64.whl (24.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.5/24.5 MB 20.8 MB/s eta
0:00:00a 0:00:01
art_open-7.5.1-py3-none-any.whl (64 kB)
Downloading wrapt-2.1.2-cp311-cp311-macosx_11_0_arm64.whl (61 kB)
Installing collected packages: wrapt, smart_open, gensim
Successfully installed gensim-4.4.0 smart_open-7.5.1 wrapt-2.1.2

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
Collecting plotly
  Downloading plotly-6.6.0-py3-none-any.whl.metadata (8.5 kB)
Collecting narwhals>=1.15.1 (from plotly)
  Downloading narwhals-2.17.0-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: packaging in
./.venv/lib/python3.11/site-packages (from plotly) (26.0)
Downloading plotly-6.6.0-py3-none-any.whl (9.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.9/9.9 MB 57.4 MB/s eta
0:00:00
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

```python
import gensim.downloader as api

# Descargamos el modelo pre-entrenado de Google News (300 dimensiones)
wv = api.load('word2vec-google-news-300')
```

4.1 Explorar el Vocabulario

Podemos ver las primeras palabras del vocabulario del modelo:

```python
# Mostramos las primeras 10 palabras del vocabulario
for index, word in enumerate(wv.index_to_key):
    if index == 10:
        break
    print(f"Palabra #{index}/{len(wv.index_to_key)}: {word}")
```

4.2 Obtener el Vector de una Palabra

Cada palabra está representada por un vector de 300 dimensiones:

```python
# Obtenemos el vector de la palabra 'king'
vec_king = wv['king']
print(f"Dimensiones del vector: {vec_king.shape}")
print(f"Primeros 10 valores: {vec_king[:10]}")
```

4.3 Palabras Desconocidas

Una limitación de Word2Vec es que **no puede generar vectores para palabras que no están en su vocabulario**. Si necesitas manejar palabras desconocidas, considera usar **FastText**, que trabaja con sub-palabras.

```
# Intentamos obtener el vector de una palabra que no existe en el
modelo
try:
    vec_cameroon = wv['cameroon']
except KeyError:
    print("La palabra 'cameroon' no existe en este modelo")
```

4.4 Similitud entre Palabras

Word2Vec nos permite calcular la **similitud coseno** entre pares de palabras. Observa cómo la similitud disminuye intuitivamente a medida que las palabras son menos relacionadas:

```
# Comparamos la similitud entre diferentes pares de palabras
pares = [
    ('car', 'minivan'),    # una minivan es un tipo de auto
    ('car', 'bicycle'),    # aún es un vehículo con ruedas
    ('car', 'airplane'),   # un vehículo, pero sin ruedas
    ('car', 'cereal'),     # sin relación aparente
    ('car', 'communism'),  # conceptos totalmente diferentes
]
for w1, w2 in pares:
    print(f'{w1:15s} {w2:15s} similitud: {wv.similarity(w1, w2):.4f}')
```

4.5 Palabras Más Similares

Podemos encontrar las palabras más cercanas a un concepto dado:

```
# Las 5 palabras más similares a 'car' y 'minivan'
print("Palabras más similares a 'car' + 'minivan':")
for palabra, similitud in wv.most_similar(positive=['car', 'minivan'],
topn=5):
    print(f"  {palabra}: {similitud:.4f}")
```

4.6 Detección de Intrusos

Word2Vec puede identificar qué palabra **no pertenece** a un grupo:

```
# ¿Cuál de estas palabras no encaja con las demás?
intruso = wv.doesnt_match(['fire', 'water', 'land', 'sea', 'air',
'car'])
print(f"La palabra que no pertenece al grupo es: '{intruso}'")
```

5. Entrenar Tu Propio Modelo

Ahora viene la parte más importante: **entrenar un modelo Word2Vec con tus propios datos**.

5.1 Preparar los Datos

Word2Vec necesita como entrada un iterable de oraciones, donde cada oración es una lista de palabras (tokens). Usaremos el **Lee Evaluation Corpus** incluido en Gensim.

Implementamos un iterador que lee el corpus línea por línea, lo cual es eficiente en memoria para corpus grandes:

```
from gensim.test.utils import datapath
from gensim import utils

class MiCorpus:
    """Iterador que produce oraciones (listas de palabras)."""

    def __iter__(self):
        ruta_corpus = datapath('lee_background.cor')
        for linea in open(ruta_corpus):
            # Asumimos un documento por línea, tokens separados por
espacios
            yield utils.simple_preprocess(linea)
```

5.2 Entrenar el Modelo

Entrenar un modelo Word2Vec con Gensim es sorprendentemente simple. Todo el preprocesamiento personalizado (minúsculas, eliminación de números, etc.) se puede hacer dentro del iterador — Word2Vec solo necesita que la entrada produzca listas de palabras.

```
import gensim.models

# Creamos el iterador del corpus
oraciones = MiCorpus()

# Entrenamos el modelo Word2Vec
modelo = gensim.models.Word2Vec(sentences=oraciones)
print("¡Modelo entrenado exitosamente!")
```

```
2026-03-08 17:45:07,402 : INFO : collecting all words and their counts
2026-03-08 17:45:07,403 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:45:07,439 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:45:07,439 : INFO : Creating a fresh vocabulary
2026-03-08 17:45:07,441 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 retains 1750 unique words (25.07% of original
6981, drops 5231)', 'datetime': '2026-03-08T17:45:07.441908',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
```

```
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:45:07,442 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 leaves 49335 word corpus (84.84% of original
58152, drops 8817)', 'datetime': '2026-03-08T17:45:07.442137',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:45:07,444 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:45:07,444 : INFO : sample=0.001 downsamples 51 most-
common words
2026-03-08 17:45:07,444 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 35935.33721568072 word corpus (72.8%%
of prior 49335)', 'datetime': '2026-03-08T17:45:07.444833', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:45:07,448 : INFO : estimated required memory for 1750
words and 100 dimensions: 2275000 bytes
2026-03-08 17:45:07,449 : INFO : resetting layer weights
2026-03-08 17:45:07,450 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:45:07.450295',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:45:07,450 : INFO : Word2Vec lifecycle event {'msg':
'training model with 3 workers on 1750 vocabulary and 100 features,
using sg=0 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
'datetime': '2026-03-08T17:45:07.450512', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:45:07,485 : INFO : EPOCH 0: training on 58152 raw words
(35865 effective words) took 0.0s, 1037514 effective words/s
2026-03-08 17:45:07,517 : INFO : EPOCH 1: training on 58152 raw words
(35873 effective words) took 0.0s, 1166178 effective words/s
2026-03-08 17:45:07,548 : INFO : EPOCH 2: training on 58152 raw words
(36004 effective words) took 0.0s, 1197071 effective words/s
2026-03-08 17:45:07,578 : INFO : EPOCH 3: training on 58152 raw words
(35962 effective words) took 0.0s, 1216611 effective words/s
2026-03-08 17:45:07,609 : INFO : EPOCH 4: training on 58152 raw words
(35944 effective words) took 0.0s, 1211187 effective words/s
2026-03-08 17:45:07,609 : INFO : Word2Vec lifecycle event {'msg':
'training on 290760 raw words (179648 effective words) took 0.2s,
1130248 effective words/s', 'datetime': '2026-03-08T17:45:07.609630',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
```

```
2026-03-08 17:45:07,609 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=1750, vector_size=100, alpha=0.025>', 'datetime':
'2026-03-08T17:45:07.609890', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}

¡Modelo entrenado exitosamente!
```

5.3 Usar el Modelo Entrenado

Una vez entrenado, podemos usar nuestro modelo de la misma forma que el modelo pre-entrenado. Los vectores de palabras están en `modelo.wv` ("wv" = *word vectors*).

```python
# Obtener el vector de una palabra
vec_king = modelo.wv['king']
print(f"Vector de 'king' (primeros 10 valores): {vec_king[:10]}")

# Ver las primeras 10 palabras del vocabulario
print("\nPrimeras 10 palabras del vocabulario:")
for index, word in enumerate(modelo.wv.index_to_key):
    if index == 10:
        break
    print(f"  #{index}: {word}")
```

```
Vector de 'king' (primeros 10 valores): [-0.01214365  0.05036781 -
0.00366184  0.01512038  0.02617095 -0.09904876
  0.01713856  0.13135819 -0.02887805 -0.01790295]

Primeras 10 palabras del vocabulario:
  #0: the
  #1: to
  #2: of
  #3: in
  #4: and
  #5: he
  #6: is
  #7: for
  #8: on
  #9: said
```

6. Guardar y Cargar Modelos

Entrenar modelos puede tomar tiempo, así que es importante poder **guardarlos en disco** y reutilizarlos después sin volver a entrenar.

```python
import tempfile

with tempfile.NamedTemporaryFile(prefix='modelo-gensim-',
```

```
delete=False) as tmp:
    ruta_temporal = tmp.name
    # Guardar el modelo en disco
    modelo.save(ruta_temporal)
    print(f"Modelo guardado en: {ruta_temporal}")

    # Cargar el modelo desde disco
    modelo_cargado = gensim.models.Word2Vec.load(ruta_temporal)
    print("Modelo cargado exitosamente")
```

```
2026-03-08 17:52:06,250 : INFO : Word2Vec lifecycle event
{'fname_or_handle':
'/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws', 'separately': 'None', 'sep_limit': 10485760, 'ignore':
frozenset(), 'datetime': '2026-03-08T17:52:06.250281', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'saving'}
2026-03-08 17:52:06,250 : INFO : not storing attribute cum_table
2026-03-08 17:52:06,251 : INFO : saved
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws
2026-03-08 17:52:06,252 : INFO : loading Word2Vec object from
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws
2026-03-08 17:52:06,253 : INFO : loading wv recursively from
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws.wv.* with mmap=None
2026-03-08 17:52:06,253 : INFO : setting ignored attribute cum_table
to None
2026-03-08 17:52:06,256 : INFO : Word2Vec lifecycle event {'fname':
'/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws', 'datetime': '2026-03-08T17:52:06.256706', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'loaded'}

Modelo guardado en:
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws
Modelo cargado exitosamente
```

También es posible cargar modelos creados con la herramienta original en C:

```
# Formato texto
modelo =
gensim.models.KeyedVectors.load_word2vec_format('/ruta/vectores.txt',
binary=False)

# Formato binario (también acepta archivos comprimidos .gz o .bz2)
```

```
modelo =
gensim.models.KeyedVectors.load_word2vec_format('/ruta/vectores.bin.gz
', binary=True)
```

7. Parámetros de Entrenamiento

Word2Vec acepta varios parámetros que afectan tanto la velocidad como la calidad del entrenamiento. Entender estos parámetros es clave para obtener buenos resultados.

7.1 `min_count` — Frecuencia Mínima

Controla el filtrado del vocabulario. Las palabras que aparecen menos de `min_count` veces se ignoran. Esto elimina errores tipográficos y palabras demasiado raras para las cuales no hay suficientes datos de entrenamiento.

- **Valor por defecto:** 5

```
# Solo considerar palabras que aparecen al menos 10 veces
modelo_mc = gensim.models.Word2Vec(oraciones, min_count=10)
print(f"Tamaño del vocabulario con min_count=10: {len(modelo_mc.wv)}")
```

```
2026-03-08 17:56:06,860 : INFO : collecting all words and their counts
2026-03-08 17:56:06,861 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:56:06,897 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:56:06,897 : INFO : Creating a fresh vocabulary
2026-03-08 17:56:06,899 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=10 retains 889 unique words (12.73% of original
6981, drops 6092)', 'datetime': '2026-03-08T17:56:06.899181',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:06,899 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=10 leaves 43776 word corpus (75.28% of original
58152, drops 14376)', 'datetime': '2026-03-08T17:56:06.899397',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:06,900 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:56:06,901 : INFO : sample=0.001 downsamples 55 most-
common words
2026-03-08 17:56:06,901 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 29691.39528319831 word corpus (67.8%%
of prior 43776)', 'datetime': '2026-03-08T17:56:06.901271', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:06,903 : INFO : estimated required memory for 889
```

```
words and 100 dimensions: 1155700 bytes
2026-03-08 17:56:06,903 : INFO : resetting layer weights
2026-03-08 17:56:06,904 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:56:06.904151',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:56:06,904 : INFO : Word2Vec lifecycle event {'msg':
'training model with 3 workers on 889 vocabulary and 100 features,
using sg=0 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
'datetime': '2026-03-08T17:56:06.904378', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:56:06,940 : INFO : EPOCH 0: training on 58152 raw words
(29626 effective words) took 0.0s, 838827 effective words/s
2026-03-08 17:56:06,972 : INFO : EPOCH 1: training on 58152 raw words
(29678 effective words) took 0.0s, 937429 effective words/s
2026-03-08 17:56:07,002 : INFO : EPOCH 2: training on 58152 raw words
(29703 effective words) took 0.0s, 1026686 effective words/s
2026-03-08 17:56:07,031 : INFO : EPOCH 3: training on 58152 raw words
(29642 effective words) took 0.0s, 1043353 effective words/s
2026-03-08 17:56:07,061 : INFO : EPOCH 4: training on 58152 raw words
(29659 effective words) took 0.0s, 1021356 effective words/s
2026-03-08 17:56:07,061 : INFO : Word2Vec lifecycle event {'msg':
'training on 290760 raw words (148308 effective words) took 0.2s,
943921 effective words/s', 'datetime': '2026-03-08T17:56:07.061633',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:56:07,061 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=889, vector_size=100, alpha=0.025>', 'datetime':
'2026-03-08T17:56:07.061829', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}

Tamaño del vocabulario con min_count=10: 889
```

7.2 `vector_size` — Dimensión de los Vectores

Define el número de dimensiones del espacio vectorial. Vectores más grandes pueden capturar relaciones más complejas, pero requieren más datos de entrenamiento y más memoria.

- **Valor por defecto:** 100
- **Valores típicos:** entre 50 y 300

```python
# Usar vectores de 200 dimensiones
modelo_vs = gensim.models.Word2Vec(oraciones, vector_size=200)
print(f"Dimensión de los vectores: {modelo_vs.wv.vector_size}")
```

```
2026-03-08 17:56:17,124 : INFO : collecting all words and their counts
2026-03-08 17:56:17,125 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:56:17,155 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:56:17,155 : INFO : Creating a fresh vocabulary
2026-03-08 17:56:17,157 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 retains 1750 unique words (25.07% of original
6981, drops 5231)', 'datetime': '2026-03-08T17:56:17.157553',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:17,157 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 leaves 49335 word corpus (84.84% of original
58152, drops 8817)', 'datetime': '2026-03-08T17:56:17.157783',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:17,159 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:56:17,160 : INFO : sample=0.001 downsamples 51 most-
common words
2026-03-08 17:56:17,160 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 35935.33721568072 word corpus (72.8%%
of prior 49335)', 'datetime': '2026-03-08T17:56:17.160324', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:17,163 : INFO : estimated required memory for 1750
words and 200 dimensions: 3675000 bytes
2026-03-08 17:56:17,164 : INFO : resetting layer weights
2026-03-08 17:56:17,166 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:56:17.166212',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:56:17,166 : INFO : Word2Vec lifecycle event {'msg':
'training model with 3 workers on 1750 vocabulary and 200 features,
using sg=0 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
'datetime': '2026-03-08T17:56:17.166380', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:56:17,203 : INFO : EPOCH 0: training on 58152 raw words
(35994 effective words) took 0.0s, 995241 effective words/s
2026-03-08 17:56:17,238 : INFO : EPOCH 1: training on 58152 raw words
(35921 effective words) took 0.0s, 1036814 effective words/s
2026-03-08 17:56:17,273 : INFO : EPOCH 2: training on 58152 raw words
(36007 effective words) took 0.0s, 1074215 effective words/s
2026-03-08 17:56:17,306 : INFO : EPOCH 3: training on 58152 raw words
```

```
(35891 effective words) took 0.0s, 1094910 effective words/s
2026-03-08 17:56:17,340 : INFO : EPOCH 4: training on 58152 raw words
(35876 effective words) took 0.0s, 1075749 effective words/s
2026-03-08 17:56:17,340 : INFO : Word2Vec lifecycle event {'msg':
'training on 290760 raw words (179689 effective words) took 0.2s,
1030277 effective words/s', 'datetime': '2026-03-08T17:56:17.340921',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:56:17,341 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=1750, vector_size=200, alpha=0.025>', 'datetime':
'2026-03-08T17:56:17.341105', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}

Dimensión de los vectores: 200
```

7.3 `sg` — Seleccionar la Arquitectura

Este parámetro selecciona entre las dos arquitecturas de Word2Vec:

- `sg=0` → **CBOW** (valor por defecto)
- `sg=1` → **Skip-gram**

```python
# Entrenar con Skip-gram
modelo_sg = gensim.models.Word2Vec(oraciones, sg=1)
print("Modelo entrenado con arquitectura Skip-gram")
```

```
2026-03-08 17:56:23,757 : INFO : collecting all words and their counts
2026-03-08 17:56:23,758 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:56:23,789 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:56:23,789 : INFO : Creating a fresh vocabulary
2026-03-08 17:56:23,792 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 retains 1750 unique words (25.07% of original
6981, drops 5231)', 'datetime': '2026-03-08T17:56:23.792055',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:23,792 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 leaves 49335 word corpus (84.84% of original
58152, drops 8817)', 'datetime': '2026-03-08T17:56:23.792291',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:23,794 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:56:23,794 : INFO : sample=0.001 downsamples 51 most-
```

```
common words
2026-03-08 17:56:23,795 : INFO : Word2Vec lifecycle event {'msg': 'downsampling leaves estimated 35935.33721568072 word corpus (72.8%% of prior 49335)', 'datetime': '2026-03-08T17:56:23.795051', 'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:56:23,798 : INFO : estimated required memory for 1750 words and 100 dimensions: 2275000 bytes
2026-03-08 17:56:23,799 : INFO : resetting layer weights
2026-03-08 17:56:23,800 : INFO : Word2Vec lifecycle event {'update': False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:56:23.800118', 'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:56:23,800 : INFO : Word2Vec lifecycle event {'msg': 'training model with 3 workers on 1750 vocabulary and 100 features, using sg=1 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True', 'datetime': '2026-03-08T17:56:23.800454', 'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:56:23,859 : INFO : EPOCH 0: training on 58152 raw words (36012 effective words) took 0.1s, 618612 effective words/s
2026-03-08 17:56:23,909 : INFO : EPOCH 1: training on 58152 raw words (35920 effective words) took 0.0s, 723879 effective words/s
2026-03-08 17:56:23,958 : INFO : EPOCH 2: training on 58152 raw words (35808 effective words) took 0.0s, 746660 effective words/s
2026-03-08 17:56:24,010 : INFO : EPOCH 3: training on 58152 raw words (35829 effective words) took 0.1s, 703859 effective words/s
2026-03-08 17:56:24,062 : INFO : EPOCH 4: training on 58152 raw words (35938 effective words) took 0.1s, 694880 effective words/s
2026-03-08 17:56:24,062 : INFO : Word2Vec lifecycle event {'msg': 'training on 290760 raw words (179507 effective words) took 0.3s, 684506 effective words/s', 'datetime': '2026-03-08T17:56:24.062946', 'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:56:24,063 : INFO : Word2Vec lifecycle event {'params': 'Word2Vec<vocab=1750, vector_size=100, alpha=0.025>', 'datetime': '2026-03-08T17:56:24.063175', 'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event': 'created'}

Modelo entrenado con arquitectura Skip-gram
```

7.4 `workers` — Paralelización

Controla el número de hilos de CPU utilizados durante el entrenamiento. Más hilos = entrenamiento más rápido (requiere Cython instalado).

- **Valor por defecto:** 3

```
# Entrenar usando 4 hilos
modelo_w = gensim.models.Word2Vec(oraciones, workers=4)
print("Modelo entrenado con 4 hilos de CPU")
```

```
2026-03-08 17:57:25,994 : INFO : collecting all words and their counts
2026-03-08 17:57:25,995 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:57:26,026 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:57:26,027 : INFO : Creating a fresh vocabulary
2026-03-08 17:57:26,029 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 retains 1750 unique words (25.07% of original
6981, drops 5231)', 'datetime': '2026-03-08T17:57:26.029309',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:57:26,029 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=5 leaves 49335 word corpus (84.84% of original
58152, drops 8817)', 'datetime': '2026-03-08T17:57:26.029544',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:57:26,031 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:57:26,032 : INFO : sample=0.001 downsamples 51 most-
common words
2026-03-08 17:57:26,032 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 35935.33721568072 word corpus (72.8%%
of prior 49335)', 'datetime': '2026-03-08T17:57:26.032181', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:57:26,035 : INFO : estimated required memory for 1750
words and 100 dimensions: 2275000 bytes
2026-03-08 17:57:26,036 : INFO : resetting layer weights
2026-03-08 17:57:26,037 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:57:26.037449',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:57:26,037 : INFO : Word2Vec lifecycle event {'msg':
'training model with 4 workers on 1750 vocabulary and 100 features,
using sg=0 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
```

```
'datetime': '2026-03-08T17:57:26.037680', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:57:26,072 : INFO : EPOCH 0: training on 58152 raw words
(35841 effective words) took 0.0s, 1060613 effective words/s
2026-03-08 17:57:26,105 : INFO : EPOCH 1: training on 58152 raw words
(35982 effective words) took 0.0s, 1157302 effective words/s
2026-03-08 17:57:26,136 : INFO : EPOCH 2: training on 58152 raw words
(36017 effective words) took 0.0s, 1193978 effective words/s
2026-03-08 17:57:26,166 : INFO : EPOCH 3: training on 58152 raw words
(36010 effective words) took 0.0s, 1206554 effective words/s
2026-03-08 17:57:26,195 : INFO : EPOCH 4: training on 58152 raw words
(35960 effective words) took 0.0s, 1286437 effective words/s
2026-03-08 17:57:26,195 : INFO : Word2Vec lifecycle event {'msg':
'training on 290760 raw words (179810 effective words) took 0.2s,
1137672 effective words/s', 'datetime': '2026-03-08T17:57:26.195877',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:57:26,196 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=1750, vector_size=100, alpha=0.025>', 'datetime':
'2026-03-08T17:57:26.196069', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}

Modelo entrenado con 4 hilos de CPU
```

7.5 Uso de Memoria

La memoria requerida por Word2Vec depende de dos factores:

- **Tamaño del vocabulario** (controlado por `min_count`)
- **Dimensión de los vectores** (`vector_size`)

La fórmula aproximada es:

$$\text{Memoria} \approx \text{vocabulario} \times \text{vector_size} \times 4\,\text{bytes} \times 3\,\text{matrices}$$

Por ejemplo, con 100,000 palabras y `vector_size=200`:

$$100{,}000 \times 200 \times 4 \times 3 = 240\,\text{MB}$$

8. Evaluación del Modelo

Word2Vec es un modelo **no supervisado**, por lo que no existe una métrica universal para evaluarlo. Sin embargo, hay dos enfoques comunes:

8.1 Analogías de Palabras

Google publicó un conjunto de pruebas con ~20,000 analogías sintácticas y semánticas del tipo *"A es a B como C es a D"*:

- **Sintácticas:** `malo:peor :: bueno:?` → mejor
- **Semánticas:** `París:Francia :: Tokio:?` → Japón

```
# Evaluar el modelo con analogías de palabras
resultados_analogias =
modelo.wv.evaluate_word_analogies(datapath('questions-words.txt'))
print(f"Precisión en analogías: {resultados_analogias[0]:.4f}")
```

```
2026-03-08 17:57:46,973 : INFO : Evaluating word analogies for top
300000 words in the model on
/Users/bryan.guzman/Documents/Personal/Universidad/2026-01/Sistemas de
Recomendación/SR-UNIANDES/Laboratorios/Laboratorio
2/.venv/lib/python3.11/site-packages/gensim/test/test_data/questions-
words.txt
2026-03-08 17:57:46,975 : INFO : capital-common-countries: 0.0% (0/6)
2026-03-08 17:57:46,981 : INFO : capital-world: 0.0% (0/2)
2026-03-08 17:57:46,985 : INFO : family: 0.0% (0/6)
2026-03-08 17:57:46,989 : INFO : gram3-comparative: 0.0% (0/20)
2026-03-08 17:57:46,991 : INFO : gram4-superlative: 0.0% (0/12)
2026-03-08 17:57:46,993 : INFO : gram5-present-participle: 0.0% (0/20)
2026-03-08 17:57:46,995 : INFO : gram6-nationality-adjective: 0.0%
(0/30)
2026-03-08 17:57:46,998 : INFO : gram7-past-tense: 0.0% (0/20)
2026-03-08 17:57:47,001 : INFO : gram8-plural: 0.0% (0/30)
2026-03-08 17:57:47,001 : INFO : Quadruplets with out-of-vocabulary
words: 99.3%
2026-03-08 17:57:47,002 : INFO : NB: analogies containing OOV words
were skipped from evaluation! To change this behavior, use
"dummy4unknown=True"
2026-03-08 17:57:47,002 : INFO : Total accuracy: 0.0% (0/146)

Precisión en analogías: 0.0000
```

8.2 Similitud entre Pares de Palabras

Otro enfoque utiliza el dataset **WS-353**, que contiene pares de palabras con puntuaciones de similitud asignadas por humanos. Medimos qué tan bien las similitudes del modelo correlacionan con los juicios humanos.

```
# Evaluar con el dataset de similitud WS-353
resultados_pares =
modelo.wv.evaluate_word_pairs(datapath('wordsim353.tsv'))
print(f"Correlación de Pearson: {resultados_pares[0][0]:.4f}")
print(f"Correlación de Spearman: {resultados_pares[1][0]:.4f}")
```

```
2026-03-08 17:57:58,355 : INFO : Skipping line #2 with OOV words: love
      sex   6.77
2026-03-08 17:57:58,355 : INFO : Skipping line #3 with OOV words:
tiger cat   7.35
2026-03-08 17:57:58,355 : INFO : Skipping line #4 with OOV words:
tiger tiger 10.00
2026-03-08 17:57:58,356 : INFO : Skipping line #5 with OOV words: book
      paper 7.46
2026-03-08 17:57:58,356 : INFO : Skipping line #6 with OOV words:
computer   keyboard   7.62
2026-03-08 17:57:58,356 : INFO : Skipping line #7 with OOV words:
computer   internet   7.58
2026-03-08 17:57:58,356 : INFO : Skipping line #9 with OOV words:
train car   6.31
2026-03-08 17:57:58,357 : INFO : Skipping line #10 with OOV words:
telephone  communication   7.50
2026-03-08 17:57:58,357 : INFO : Skipping line #14 with OOV words:
bread butter     6.19
2026-03-08 17:57:58,358 : INFO : Skipping line #15 with OOV words:
cucumber   potato     5.92
2026-03-08 17:57:58,358 : INFO : Skipping line #16 with OOV words:
doctor     nurse 7.00
2026-03-08 17:57:58,358 : INFO : Skipping line #18 with OOV words:
student    professor  6.81
2026-03-08 17:57:58,358 : INFO : Skipping line #19 with OOV words:
smart student    4.62
2026-03-08 17:57:58,359 : INFO : Skipping line #20 with OOV words:
smart stupid     5.81
2026-03-08 17:57:58,359 : INFO : Skipping line #21 with OOV words:
company    stock 7.08
2026-03-08 17:57:58,359 : INFO : Skipping line #22 with OOV words:
stock market     8.08
2026-03-08 17:57:58,359 : INFO : Skipping line #23 with OOV words:
stock phone 1.62
2026-03-08 17:57:58,360 : INFO : Skipping line #24 with OOV words:
stock CD    1.31
2026-03-08 17:57:58,360 : INFO : Skipping line #25 with OOV words:
stock jaguar     0.92
2026-03-08 17:57:58,360 : INFO : Skipping line #26 with OOV words:
stock egg   1.81
2026-03-08 17:57:58,360 : INFO : Skipping line #27 with OOV words:
fertility  egg   6.69
2026-03-08 17:57:58,360 : INFO : Skipping line #28 with OOV words:
stock live  3.73
2026-03-08 17:57:58,361 : INFO : Skipping line #29 with OOV words:
stock life  0.92
2026-03-08 17:57:58,361 : INFO : Skipping line #30 with OOV words:
book  library    7.46
2026-03-08 17:57:58,361 : INFO : Skipping line #32 with OOV words:
wood  forest     7.73
```

```
2026-03-08 17:57:58,361 : INFO : Skipping line #33 with OOV words:
money cash  9.15
2026-03-08 17:57:58,361 : INFO : Skipping line #34 with OOV words:
professor  cucumber   0.31
2026-03-08 17:57:58,362 : INFO : Skipping line #35 with OOV words:
king  cabbage    0.23
2026-03-08 17:57:58,362 : INFO : Skipping line #36 with OOV words:
king  queen 8.58
2026-03-08 17:57:58,362 : INFO : Skipping line #37 with OOV words:
king  rook  5.92
2026-03-08 17:57:58,362 : INFO : Skipping line #38 with OOV words:
bishop     rabbi 6.69
2026-03-08 17:57:58,362 : INFO : Skipping line #41 with OOV words:
holy  sex   1.62
2026-03-08 17:57:58,362 : INFO : Skipping line #42 with OOV words:
fuck  sex   9.44
2026-03-08 17:57:58,362 : INFO : Skipping line #43 with OOV words:
Maradona   football   8.62
2026-03-08 17:57:58,362 : INFO : Skipping line #44 with OOV words:
football   soccer     9.03
2026-03-08 17:57:58,363 : INFO : Skipping line #45 with OOV words:
football   basketball 6.81
2026-03-08 17:57:58,363 : INFO : Skipping line #46 with OOV words:
football   tennis     6.63
2026-03-08 17:57:58,363 : INFO : Skipping line #47 with OOV words:
tennis     racket     7.56
2026-03-08 17:57:58,363 : INFO : Skipping line #50 with OOV words:
Arafat     Jackson    2.50
2026-03-08 17:57:58,363 : INFO : Skipping line #51 with OOV words: law
     lawyer     8.38
2026-03-08 17:57:58,363 : INFO : Skipping line #52 with OOV words:
movie star  7.38
2026-03-08 17:57:58,364 : INFO : Skipping line #53 with OOV words:
movie popcorn    6.19
2026-03-08 17:57:58,364 : INFO : Skipping line #54 with OOV words:
movie critic     6.73
2026-03-08 17:57:58,364 : INFO : Skipping line #55 with OOV words:
movie theater    7.92
2026-03-08 17:57:58,364 : INFO : Skipping line #56 with OOV words:
physics    proton     8.12
2026-03-08 17:57:58,364 : INFO : Skipping line #57 with OOV words:
physics    chemistry  7.35
2026-03-08 17:57:58,364 : INFO : Skipping line #58 with OOV words:
space chemistry  4.88
2026-03-08 17:57:58,364 : INFO : Skipping line #59 with OOV words:
alcohol    chemistry  5.54
2026-03-08 17:57:58,364 : INFO : Skipping line #60 with OOV words:
vodka gin   8.46
2026-03-08 17:57:58,365 : INFO : Skipping line #61 with OOV words:
vodka brandy     8.13
```

```
2026-03-08 17:57:58,365 : INFO : Skipping line #62 with OOV words:
drink car   3.04
2026-03-08 17:57:58,365 : INFO : Skipping line #63 with OOV words:
drink ear   1.31
2026-03-08 17:57:58,365 : INFO : Skipping line #64 with OOV words:
drink mouth 5.96
2026-03-08 17:57:58,365 : INFO : Skipping line #65 with OOV words:
drink eat   6.87
2026-03-08 17:57:58,365 : INFO : Skipping line #66 with OOV words:
baby  mother     7.85
2026-03-08 17:57:58,365 : INFO : Skipping line #67 with OOV words:
drink mother     2.65
2026-03-08 17:57:58,366 : INFO : Skipping line #68 with OOV words: car
     automobile 8.94
2026-03-08 17:57:58,366 : INFO : Skipping line #69 with OOV words: gem
     jewel 8.96
2026-03-08 17:57:58,366 : INFO : Skipping line #70 with OOV words:
journey    voyage     9.29
2026-03-08 17:57:58,367 : INFO : Skipping line #71 with OOV words: boy
     lad   8.83
2026-03-08 17:57:58,367 : INFO : Skipping line #72 with OOV words:
coast shore 9.10
2026-03-08 17:57:58,367 : INFO : Skipping line #73 with OOV words:
asylum     madhouse   8.87
2026-03-08 17:57:58,367 : INFO : Skipping line #74 with OOV words:
magician   wizard     9.02
2026-03-08 17:57:58,367 : INFO : Skipping line #75 with OOV words:
midday     noon  9.29
2026-03-08 17:57:58,367 : INFO : Skipping line #76 with OOV words:
furnace    stove 8.79
2026-03-08 17:57:58,367 : INFO : Skipping line #77 with OOV words:
food  fruit 7.52
2026-03-08 17:57:58,368 : INFO : Skipping line #78 with OOV words:
bird  cock  7.10
2026-03-08 17:57:58,368 : INFO : Skipping line #79 with OOV words:
bird  crane 7.38
2026-03-08 17:57:58,368 : INFO : Skipping line #80 with OOV words:
tool  implement  6.46
2026-03-08 17:57:58,368 : INFO : Skipping line #81 with OOV words:
brother    monk  6.27
2026-03-08 17:57:58,368 : INFO : Skipping line #82 with OOV words:
crane implement  2.69
2026-03-08 17:57:58,368 : INFO : Skipping line #83 with OOV words: lad
     brother    4.46
2026-03-08 17:57:58,368 : INFO : Skipping line #84 with OOV words:
journey    car   5.85
2026-03-08 17:57:58,368 : INFO : Skipping line #85 with OOV words:
monk  oracle     5.00
2026-03-08 17:57:58,369 : INFO : Skipping line #86 with OOV words:
cemetery   woodland   2.08
```

```
2026-03-08 17:57:58,369 : INFO : Skipping line #87 with OOV words:
food  rooster    4.42
2026-03-08 17:57:58,369 : INFO : Skipping line #89 with OOV words:
forest     graveyard  1.85
2026-03-08 17:57:58,369 : INFO : Skipping line #90 with OOV words:
shore woodland   3.08
2026-03-08 17:57:58,369 : INFO : Skipping line #91 with OOV words:
monk  slave 0.92
2026-03-08 17:57:58,369 : INFO : Skipping line #92 with OOV words:
coast forest     3.15
2026-03-08 17:57:58,370 : INFO : Skipping line #93 with OOV words: lad
     wizard     0.92
2026-03-08 17:57:58,370 : INFO : Skipping line #94 with OOV words:
chord smile 0.54
2026-03-08 17:57:58,370 : INFO : Skipping line #95 with OOV words:
glass magician   2.08
2026-03-08 17:57:58,370 : INFO : Skipping line #96 with OOV words:
noon  string     0.54
2026-03-08 17:57:58,370 : INFO : Skipping line #97 with OOV words:
rooster    voyage     0.62
2026-03-08 17:57:58,370 : INFO : Skipping line #98 with OOV words:
money dollar     8.42
2026-03-08 17:57:58,370 : INFO : Skipping line #99 with OOV words:
money cash  9.08
2026-03-08 17:57:58,370 : INFO : Skipping line #100 with OOV words:
money currency   9.04
2026-03-08 17:57:58,371 : INFO : Skipping line #101 with OOV words:
money wealth     8.27
2026-03-08 17:57:58,371 : INFO : Skipping line #103 with OOV words:
money possession 7.29
2026-03-08 17:57:58,371 : INFO : Skipping line #105 with OOV words:
money deposit    7.73
2026-03-08 17:57:58,371 : INFO : Skipping line #106 with OOV words:
money withdrawal 6.88
2026-03-08 17:57:58,371 : INFO : Skipping line #107 with OOV words:
money laundering 5.65
2026-03-08 17:57:58,371 : INFO : Skipping line #109 with OOV words:
tiger jaguar     8.00
2026-03-08 17:57:58,371 : INFO : Skipping line #110 with OOV words:
tiger feline     8.00
2026-03-08 17:57:58,371 : INFO : Skipping line #111 with OOV words:
tiger carnivore  7.08
2026-03-08 17:57:58,372 : INFO : Skipping line #112 with OOV words:
tiger mammal     6.85
2026-03-08 17:57:58,372 : INFO : Skipping line #113 with OOV words:
tiger animal     7.00
2026-03-08 17:57:58,372 : INFO : Skipping line #114 with OOV words:
tiger organism   4.77
2026-03-08 17:57:58,372 : INFO : Skipping line #115 with OOV words:
tiger fauna 5.62
```

```
2026-03-08 17:57:58,372 : INFO : Skipping line #116 with OOV words:
tiger zoo   5.87
2026-03-08 17:57:58,372 : INFO : Skipping line #117 with OOV words:
psychology psychiatry 8.08
2026-03-08 17:57:58,372 : INFO : Skipping line #118 with OOV words:
psychology anxiety    7.00
2026-03-08 17:57:58,373 : INFO : Skipping line #119 with OOV words:
psychology fear  6.85
2026-03-08 17:57:58,373 : INFO : Skipping line #120 with OOV words:
psychology depression 7.42
2026-03-08 17:57:58,373 : INFO : Skipping line #121 with OOV words:
psychology clinic     6.58
2026-03-08 17:57:58,373 : INFO : Skipping line #122 with OOV words:
psychology doctor     6.42
2026-03-08 17:57:58,373 : INFO : Skipping line #123 with OOV words:
psychology Freud 8.21
2026-03-08 17:57:58,373 : INFO : Skipping line #124 with OOV words:
psychology mind  7.69
2026-03-08 17:57:58,374 : INFO : Skipping line #125 with OOV words:
psychology health     7.23
2026-03-08 17:57:58,374 : INFO : Skipping line #126 with OOV words:
psychology science    6.71
2026-03-08 17:57:58,374 : INFO : Skipping line #127 with OOV words:
psychology discipline 5.58
2026-03-08 17:57:58,374 : INFO : Skipping line #128 with OOV words:
psychology cognition  7.48
2026-03-08 17:57:58,374 : INFO : Skipping line #129 with OOV words:
planet     star  8.45
2026-03-08 17:57:58,375 : INFO : Skipping line #130 with OOV words:
planet     constellation   8.06
2026-03-08 17:57:58,375 : INFO : Skipping line #131 with OOV words:
planet     moon  8.08
2026-03-08 17:57:58,375 : INFO : Skipping line #132 with OOV words:
planet     sun   8.02
2026-03-08 17:57:58,375 : INFO : Skipping line #133 with OOV words:
planet     galaxy     8.11
2026-03-08 17:57:58,375 : INFO : Skipping line #134 with OOV words:
planet     space 7.92
2026-03-08 17:57:58,375 : INFO : Skipping line #135 with OOV words:
planet     astronomer 7.94
2026-03-08 17:57:58,375 : INFO : Skipping line #136 with OOV words:
precedent  example    5.85
2026-03-08 17:57:58,376 : INFO : Skipping line #137 with OOV words:
precedent  information     3.85
2026-03-08 17:57:58,376 : INFO : Skipping line #138 with OOV words:
precedent  cognition  2.81
2026-03-08 17:57:58,376 : INFO : Skipping line #139 with OOV words:
precedent  law   6.65
2026-03-08 17:57:58,376 : INFO : Skipping line #140 with OOV words:
precedent  collection 2.50
```

```
2026-03-08 17:57:58,376 : INFO : Skipping line #141 with OOV words:
precedent  group 1.77
2026-03-08 17:57:58,376 : INFO : Skipping line #142 with OOV words:
precedent  antecedent 6.04
2026-03-08 17:57:58,376 : INFO : Skipping line #143 with OOV words:
cup   coffee     6.58
2026-03-08 17:57:58,377 : INFO : Skipping line #144 with OOV words:
cup   tableware  6.85
2026-03-08 17:57:58,377 : INFO : Skipping line #145 with OOV words:
cup   article    2.40
2026-03-08 17:57:58,377 : INFO : Skipping line #146 with OOV words:
cup   artifact   2.92
2026-03-08 17:57:58,377 : INFO : Skipping line #147 with OOV words:
cup   object     3.69
2026-03-08 17:57:58,377 : INFO : Skipping line #148 with OOV words:
cup   entity     2.15
2026-03-08 17:57:58,377 : INFO : Skipping line #149 with OOV words:
cup   drink 7.25
2026-03-08 17:57:58,378 : INFO : Skipping line #151 with OOV words:
cup   substance  1.92
2026-03-08 17:57:58,378 : INFO : Skipping line #152 with OOV words:
cup   liquid     5.90
2026-03-08 17:57:58,378 : INFO : Skipping line #153 with OOV words:
jaguar     cat   7.42
2026-03-08 17:57:58,378 : INFO : Skipping line #154 with OOV words:
jaguar     car   7.27
2026-03-08 17:57:58,378 : INFO : Skipping line #157 with OOV words:
energy     laboratory 5.09
2026-03-08 17:57:58,379 : INFO : Skipping line #158 with OOV words:
computer   laboratory 6.78
2026-03-08 17:57:58,379 : INFO : Skipping line #159 with OOV words:
weapon     secret     6.06
2026-03-08 17:57:58,379 : INFO : Skipping line #160 with OOV words:
FBI   fingerprint     6.94
2026-03-08 17:57:58,379 : INFO : Skipping line #161 with OOV words:
FBI   investigation   8.31
2026-03-08 17:57:58,379 : INFO : Skipping line #163 with OOV words:
Mars  water 2.94
2026-03-08 17:57:58,379 : INFO : Skipping line #164 with OOV words:
Mars  scientist  5.63
2026-03-08 17:57:58,380 : INFO : Skipping line #166 with OOV words:
canyon     landscape  7.53
2026-03-08 17:57:58,380 : INFO : Skipping line #167 with OOV words:
image surface    4.56
2026-03-08 17:57:58,380 : INFO : Skipping line #168 with OOV words:
discovery  space 6.34
2026-03-08 17:57:58,380 : INFO : Skipping line #169 with OOV words:
water seepage    6.56
2026-03-08 17:57:58,380 : INFO : Skipping line #170 with OOV words:
sign  recess     2.38
```

```
2026-03-08 17:57:58,380 : INFO : Skipping line #172 with OOV words:
mile  kilometer  8.66
2026-03-08 17:57:58,381 : INFO : Skipping line #173 with OOV words:
computer   news  4.47
2026-03-08 17:57:58,381 : INFO : Skipping line #174 with OOV words:
territory  surface    5.34
2026-03-08 17:57:58,381 : INFO : Skipping line #175 with OOV words:
atmosphere landscape  3.69
2026-03-08 17:57:58,381 : INFO : Skipping line #176 with OOV words:
president  medal 3.00
2026-03-08 17:57:58,381 : INFO : Skipping line #179 with OOV words:
skin  eye   6.22
2026-03-08 17:57:58,382 : INFO : Skipping line #181 with OOV words:
theater    history    3.91
2026-03-08 17:57:58,382 : INFO : Skipping line #182 with OOV words:
volunteer  motto 2.56
2026-03-08 17:57:58,382 : INFO : Skipping line #183 with OOV words:
prejudice  recognition     3.00
2026-03-08 17:57:58,382 : INFO : Skipping line #184 with OOV words:
decoration valor 5.63
2026-03-08 17:57:58,383 : INFO : Skipping line #185 with OOV words:
century    year  7.59
2026-03-08 17:57:58,383 : INFO : Skipping line #186 with OOV words:
century    nation     3.16
2026-03-08 17:57:58,383 : INFO : Skipping line #187 with OOV words:
delay racism     1.19
2026-03-08 17:57:58,383 : INFO : Skipping line #191 with OOV words:
minority   peace 3.69
2026-03-08 17:57:58,383 : INFO : Skipping line #194 with OOV words:
deployment departure  4.25
2026-03-08 17:57:58,384 : INFO : Skipping line #195 with OOV words:
deployment withdrawal 5.88
2026-03-08 17:57:58,384 : INFO : Skipping line #197 with OOV words:
announcement    news  7.56
2026-03-08 17:57:58,384 : INFO : Skipping line #198 with OOV words:
announcement    effort     2.75
2026-03-08 17:57:58,384 : INFO : Skipping line #199 with OOV words:
stroke     hospital   7.03
2026-03-08 17:57:58,385 : INFO : Skipping line #200 with OOV words:
disability death 5.47
2026-03-08 17:57:58,385 : INFO : Skipping line #201 with OOV words:
victim     emergency  6.47
2026-03-08 17:57:58,385 : INFO : Skipping line #203 with OOV words:
journal    association     4.97
2026-03-08 17:57:58,385 : INFO : Skipping line #205 with OOV words:
doctor     liability  5.19
2026-03-08 17:57:58,385 : INFO : Skipping line #206 with OOV words:
liability  insurance  7.03
2026-03-08 17:57:58,385 : INFO : Skipping line #207 with OOV words:
school     center     3.44
```

```
2026-03-08 17:57:58,385 : INFO : Skipping line #208 with OOV words:
reason     hypertension    2.31
2026-03-08 17:57:58,385 : INFO : Skipping line #209 with OOV words:
reason     criterion  5.91
2026-03-08 17:57:58,386 : INFO : Skipping line #210 with OOV words:
hundred    percent    7.38
2026-03-08 17:57:58,386 : INFO : Skipping line #211 with OOV words:
Harvard    Yale  8.13
2026-03-08 17:57:58,386 : INFO : Skipping line #212 with OOV words:
hospital   infrastructure  4.63
2026-03-08 17:57:58,386 : INFO : Skipping line #213 with OOV words:
death row   5.25
2026-03-08 17:57:58,386 : INFO : Skipping line #214 with OOV words:
death inmate     5.03
2026-03-08 17:57:58,386 : INFO : Skipping line #215 with OOV words:
lawyer     evidence   6.69
2026-03-08 17:57:58,386 : INFO : Skipping line #218 with OOV words:
word  similarity 4.75
2026-03-08 17:57:58,386 : INFO : Skipping line #219 with OOV words:
board recommendation   4.47
2026-03-08 17:57:58,387 : INFO : Skipping line #221 with OOV words:
OPEC  country    5.63
2026-03-08 17:57:58,387 : INFO : Skipping line #222 with OOV words:
peace atmosphere 3.69
2026-03-08 17:57:58,387 : INFO : Skipping line #224 with OOV words:
territory  kilometer  5.28
2026-03-08 17:57:58,388 : INFO : Skipping line #226 with OOV words:
competition     price 6.44
2026-03-08 17:57:58,388 : INFO : Skipping line #227 with OOV words:
consumer   confidence 4.13
2026-03-08 17:57:58,388 : INFO : Skipping line #228 with OOV words:
consumer   energy     4.75
2026-03-08 17:57:58,388 : INFO : Skipping line #231 with OOV words:
credit     card  8.06
2026-03-08 17:57:58,388 : INFO : Skipping line #233 with OOV words:
hotel reservation     8.03
2026-03-08 17:57:58,389 : INFO : Skipping line #234 with OOV words:
grocery    money 5.94
2026-03-08 17:57:58,389 : INFO : Skipping line #235 with OOV words:
registration    arrangement     6.00
2026-03-08 17:57:58,389 : INFO : Skipping line #236 with OOV words:
arrangement     accommodation   5.41
2026-03-08 17:57:58,389 : INFO : Skipping line #238 with OOV words:
type  kind  8.97
2026-03-08 17:57:58,389 : INFO : Skipping line #239 with OOV words:
arrival    hotel 6.00
2026-03-08 17:57:58,389 : INFO : Skipping line #240 with OOV words:
bed   closet     6.72
2026-03-08 17:57:58,389 : INFO : Skipping line #241 with OOV words:
closet     clothes    8.00
```

```
2026-03-08 17:57:58,390 : INFO : Skipping line #242 with OOV words:
situation  conclusion 4.81
2026-03-08 17:57:58,390 : INFO : Skipping line #243 with OOV words:
situation  isolation  3.88
2026-03-08 17:57:58,390 : INFO : Skipping line #244 with OOV words:
impartiality    interest   5.16
2026-03-08 17:57:58,390 : INFO : Skipping line #245 with OOV words:
direction  combination     2.25
2026-03-08 17:57:58,390 : INFO : Skipping line #246 with OOV words:
street     place 6.44
2026-03-08 17:57:58,390 : INFO : Skipping line #247 with OOV words:
street     avenue     8.88
2026-03-08 17:57:58,391 : INFO : Skipping line #248 with OOV words:
street     block 6.88
2026-03-08 17:57:58,391 : INFO : Skipping line #249 with OOV words:
street     children   4.94
2026-03-08 17:57:58,391 : INFO : Skipping line #250 with OOV words:
listing    proximity  2.56
2026-03-08 17:57:58,391 : INFO : Skipping line #251 with OOV words:
listing    category   6.38
2026-03-08 17:57:58,391 : INFO : Skipping line #252 with OOV words:
cell  phone 7.81
2026-03-08 17:57:58,391 : INFO : Skipping line #253 with OOV words:
production hike  1.75
2026-03-08 17:57:58,391 : INFO : Skipping line #254 with OOV words:
benchmark  index 4.25
2026-03-08 17:57:58,392 : INFO : Skipping line #256 with OOV words:
media gain  2.88
2026-03-08 17:57:58,392 : INFO : Skipping line #257 with OOV words:
dividend   payment    7.63
2026-03-08 17:57:58,392 : INFO : Skipping line #258 with OOV words:
dividend   calculation     6.48
2026-03-08 17:57:58,392 : INFO : Skipping line #259 with OOV words:
calculation     computation     8.44
2026-03-08 17:57:58,392 : INFO : Skipping line #260 with OOV words:
currency   market     7.50
2026-03-08 17:57:58,392 : INFO : Skipping line #261 with OOV words:
OPEC  oil   8.59
2026-03-08 17:57:58,392 : INFO : Skipping line #262 with OOV words:
oil   stock 6.34
2026-03-08 17:57:58,393 : INFO : Skipping line #263 with OOV words:
announcement    production 3.38
2026-03-08 17:57:58,393 : INFO : Skipping line #264 with OOV words:
announcement    warning    6.00
2026-03-08 17:57:58,393 : INFO : Skipping line #265 with OOV words:
profit     warning    3.88
2026-03-08 17:57:58,393 : INFO : Skipping line #266 with OOV words:
profit     loss  7.63
2026-03-08 17:57:58,393 : INFO : Skipping line #267 with OOV words:
dollar     yen   7.78
```

```
2026-03-08 17:57:58,393 : INFO : Skipping line #268 with OOV words:
dollar     buck  9.22
2026-03-08 17:57:58,393 : INFO : Skipping line #269 with OOV words:
dollar     profit     7.38
2026-03-08 17:57:58,394 : INFO : Skipping line #270 with OOV words:
dollar     loss  6.09
2026-03-08 17:57:58,394 : INFO : Skipping line #271 with OOV words:
computer   software   8.50
2026-03-08 17:57:58,394 : INFO : Skipping line #272 with OOV words:
network    hardware   8.31
2026-03-08 17:57:58,394 : INFO : Skipping line #273 with OOV words:
phone equipment  7.13
2026-03-08 17:57:58,394 : INFO : Skipping line #274 with OOV words:
equipment  maker 5.91
2026-03-08 17:57:58,394 : INFO : Skipping line #275 with OOV words:
luxury     car   6.47
2026-03-08 17:57:58,394 : INFO : Skipping line #277 with OOV words:
report     gain  3.63
2026-03-08 17:57:58,395 : INFO : Skipping line #278 with OOV words:
investor   earning    7.13
2026-03-08 17:57:58,395 : INFO : Skipping line #279 with OOV words:
liquid     water 7.89
2026-03-08 17:57:58,395 : INFO : Skipping line #280 with OOV words:
baseball   season     5.97
2026-03-08 17:57:58,395 : INFO : Skipping line #283 with OOV words:
marathon   sprint     7.47
2026-03-08 17:57:58,396 : INFO : Skipping line #285 with OOV words:
game  defeat     6.97
2026-03-08 17:57:58,396 : INFO : Skipping line #287 with OOV words:
seafood    sea   7.47
2026-03-08 17:57:58,396 : INFO : Skipping line #288 with OOV words:
seafood    food  8.34
2026-03-08 17:57:58,396 : INFO : Skipping line #289 with OOV words:
seafood    lobster    8.70
2026-03-08 17:57:58,396 : INFO : Skipping line #290 with OOV words:
lobster    food  7.81
2026-03-08 17:57:58,396 : INFO : Skipping line #291 with OOV words:
lobster    wine  5.70
2026-03-08 17:57:58,397 : INFO : Skipping line #292 with OOV words:
food  preparation     6.22
2026-03-08 17:57:58,397 : INFO : Skipping line #293 with OOV words:
video archive    6.34
2026-03-08 17:57:58,397 : INFO : Skipping line #298 with OOV words:
championship    tournament 8.36
2026-03-08 17:57:58,397 : INFO : Skipping line #299 with OOV words:
fighting   defeating  7.41
2026-03-08 17:57:58,398 : INFO : Skipping line #301 with OOV words:
day   summer     3.94
2026-03-08 17:57:58,398 : INFO : Skipping line #302 with OOV words:
summer     drought    7.16
```

```
2026-03-08 17:57:58,398 : INFO : Skipping line #303 with OOV words:
summer     nature     5.63
2026-03-08 17:57:58,398 : INFO : Skipping line #304 with OOV words:
day   dawn  7.53
2026-03-08 17:57:58,398 : INFO : Skipping line #305 with OOV words:
nature     environment     8.31
2026-03-08 17:57:58,398 : INFO : Skipping line #306 with OOV words:
environment     ecology    8.81
2026-03-08 17:57:58,398 : INFO : Skipping line #307 with OOV words:
nature     man   6.25
2026-03-08 17:57:58,398 : INFO : Skipping line #311 with OOV words:
soap  opera 7.94
2026-03-08 17:57:58,399 : INFO : Skipping line #312 with OOV words:
opera performance     6.88
2026-03-08 17:57:58,399 : INFO : Skipping line #313 with OOV words:
life  lesson     5.94
2026-03-08 17:57:58,399 : INFO : Skipping line #315 with OOV words:
production crew  6.25
2026-03-08 17:57:58,399 : INFO : Skipping line #316 with OOV words:
television film  7.72
2026-03-08 17:57:58,399 : INFO : Skipping line #317 with OOV words:
lover quarrel    6.19
2026-03-08 17:57:58,399 : INFO : Skipping line #318 with OOV words:
viewer     serial     2.97
2026-03-08 17:57:58,399 : INFO : Skipping line #319 with OOV words:
possibility     girl  1.94
2026-03-08 17:57:58,399 : INFO : Skipping line #320 with OOV words:
population development     3.75
2026-03-08 17:57:58,400 : INFO : Skipping line #321 with OOV words:
morality   importance 3.31
2026-03-08 17:57:58,400 : INFO : Skipping line #322 with OOV words:
morality   marriage   3.69
2026-03-08 17:57:58,400 : INFO : Skipping line #323 with OOV words:
Mexico     Brazil     7.44
2026-03-08 17:57:58,400 : INFO : Skipping line #324 with OOV words:
gender     equality   6.41
2026-03-08 17:57:58,400 : INFO : Skipping line #325 with OOV words:
change     attitude   5.44
2026-03-08 17:57:58,401 : INFO : Skipping line #327 with OOV words:
opera industry   2.63
2026-03-08 17:57:58,401 : INFO : Skipping line #328 with OOV words:
sugar approach   0.88
2026-03-08 17:57:58,401 : INFO : Skipping line #329 with OOV words:
practice   institution     3.19
2026-03-08 17:57:58,401 : INFO : Skipping line #330 with OOV words:
ministry   culture    4.69
2026-03-08 17:57:58,401 : INFO : Skipping line #331 with OOV words:
problem    challenge  6.75
2026-03-08 17:57:58,402 : INFO : Skipping line #332 with OOV words:
size  prominence 5.31
```

```
2026-03-08 17:57:58,402 : INFO : Skipping line #333 with OOV words:
country    citizen    7.31
2026-03-08 17:57:58,402 : INFO : Skipping line #334 with OOV words:
planet     people     5.75
2026-03-08 17:57:58,402 : INFO : Skipping line #335 with OOV words:
development     issue 3.97
2026-03-08 17:57:58,402 : INFO : Skipping line #336 with OOV words:
experience music 3.47
2026-03-08 17:57:58,402 : INFO : Skipping line #337 with OOV words:
music project    3.63
2026-03-08 17:57:58,402 : INFO : Skipping line #338 with OOV words:
glass metal 5.56
2026-03-08 17:57:58,403 : INFO : Skipping line #339 with OOV words:
aluminum   metal 7.83
2026-03-08 17:57:58,403 : INFO : Skipping line #340 with OOV words:
chance     credibility     3.88
2026-03-08 17:57:58,403 : INFO : Skipping line #341 with OOV words:
exhibit    memorabilia     5.31
2026-03-08 17:57:58,403 : INFO : Skipping line #342 with OOV words:
concert    virtuoso   6.81
2026-03-08 17:57:58,403 : INFO : Skipping line #343 with OOV words:
rock  jazz  7.59
2026-03-08 17:57:58,403 : INFO : Skipping line #344 with OOV words:
museum     theater    7.19
2026-03-08 17:57:58,404 : INFO : Skipping line #345 with OOV words:
observation     architecture    4.38
2026-03-08 17:57:58,404 : INFO : Skipping line #347 with OOV words:
preservation    world 6.19
2026-03-08 17:57:58,404 : INFO : Skipping line #348 with OOV words:
admission  ticket     7.69
2026-03-08 17:57:58,404 : INFO : Skipping line #349 with OOV words:
shower     thunderstorm    6.31
2026-03-08 17:57:58,404 : INFO : Skipping line #350 with OOV words:
shower     flood 6.03
2026-03-08 17:57:58,404 : INFO : Skipping line #354 with OOV words:
architecture    century    3.78
2026-03-08 17:57:58,406 : INFO : Pearson correlation coefficient
against
/Users/bryan.guzman/Documents/Personal/Universidad/2026-01/Sistemas de
Recomendación/SR-UNIANDES/Laboratorios/Laboratorio
2/.venv/lib/python3.11/site-packages/gensim/test/test_data/wordsim353.
tsv: 0.2192
2026-03-08 17:57:58,406 : INFO : Spearman rank-order correlation
coefficient against
/Users/bryan.guzman/Documents/Personal/Universidad/2026-01/Sistemas de
Recomendación/SR-UNIANDES/Laboratorios/Laboratorio
2/.venv/lib/python3.11/site-packages/gensim/test/test_data/wordsim353.
tsv: 0.1557
2026-03-08 17:57:58,406 : INFO : Pairs with unknown words ratio: 83.0%
```

```
Correlación de Pearson: 0.2192
Correlación de Spearman: 0.1557
```

> **Importante:** Un buen desempeño en estos benchmarks no garantiza que el modelo funcione bien en tu tarea específica. Siempre es mejor evaluar directamente en tu aplicación final.

9. Entrenamiento Incremental

Es posible cargar un modelo existente y **continuar entrenándolo** con nuevas oraciones y vocabulario adicional:

```
# Cargar el modelo previamente guardado
modelo = gensim.models.Word2Vec.load(ruta_temporal)

# Nuevas oraciones para continuar el entrenamiento
nuevas_oraciones = [
    ['los', 'usuarios', 'avanzados', 'pueden', 'cargar', 'un',
'modelo',
     'y', 'continuar', 'entrenando', 'con', 'mas', 'oraciones'],
]

# Actualizar el vocabulario con nuevas palabras
modelo.build_vocab(nuevas_oraciones, update=True)

# Continuar el entrenamiento
modelo.train(nuevas_oraciones, total_examples=modelo.corpus_count,
epochs=modelo.epochs)
print("Entrenamiento incremental completado")

# Limpiar el archivo temporal
import os
os.remove(ruta_temporal)
```

```
2026-03-08 17:58:10,019 : INFO : loading Word2Vec object from
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws
2026-03-08 17:58:10,022 : INFO : loading wv recursively from
/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws.wv.* with mmap=None
2026-03-08 17:58:10,022 : INFO : setting ignored attribute cum_table
to None
2026-03-08 17:58:10,027 : INFO : Word2Vec lifecycle event {'fname':
'/var/folders/wl/p0mswpjx6lq14c600b71t6540000gn/T/modelo-gensim-
ikrhivws', 'datetime': '2026-03-08T17:58:10.027608', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'loaded'}
2026-03-08 17:58:10,028 : INFO : collecting all words and their counts
```

```
2026-03-08 17:58:10,028 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:58:10,028 : INFO : collected 13 word types from a corpus
of 13 raw words and 1 sentences
2026-03-08 17:58:10,028 : INFO : Updating model with new vocabulary
2026-03-08 17:58:10,030 : INFO : Word2Vec lifecycle event {'msg':
'added 0 new unique words (0.00% of original 13) and increased the
count of 0 pre-existing words (0.00% of original 13)', 'datetime':
'2026-03-08T17:58:10.030518', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'prepare_vocab'}
2026-03-08 17:58:10,031 : INFO : deleting the raw counts dictionary of
13 items
2026-03-08 17:58:10,031 : INFO : sample=0.001 downsamples 0 most-
common words
2026-03-08 17:58:10,031 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 0 word corpus (0.0%% of prior 0)',
'datetime': '2026-03-08T17:58:10.031619', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'prepare_vocab'}
2026-03-08 17:58:10,035 : INFO : estimated required memory for 1750
words and 100 dimensions: 2275000 bytes
2026-03-08 17:58:10,036 : INFO : updating layer weights
2026-03-08 17:58:10,036 : INFO : Word2Vec lifecycle event {'update':
True, 'trim_rule': 'None', 'datetime': '2026-03-08T17:58:10.036454',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:58:10,036 : WARNING : Effective 'alpha' higher than
previous training cycles
2026-03-08 17:58:10,036 : INFO : Word2Vec lifecycle event {'msg':
'training model with 3 workers on 1750 vocabulary and 100 features,
using sg=0 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
'datetime': '2026-03-08T17:58:10.036758', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:58:10,037 : INFO : EPOCH 0: training on 13 raw words (1
effective words) took 0.0s, 10444 effective words/s
2026-03-08 17:58:10,038 : INFO : EPOCH 1: training on 13 raw words (1
effective words) took 0.0s, 3739 effective words/s
2026-03-08 17:58:10,039 : INFO : EPOCH 2: training on 13 raw words (1
effective words) took 0.0s, 4208 effective words/s
2026-03-08 17:58:10,040 : INFO : EPOCH 3: training on 13 raw words (1
effective words) took 0.0s, 6439 effective words/s
2026-03-08 17:58:10,041 : INFO : EPOCH 4: training on 13 raw words (1
effective words) took 0.0s, 5215 effective words/s
```

```
2026-03-08 17:58:10,041 : INFO : Word2Vec lifecycle event {'msg':
'training on 65 raw words (5 effective words) took 0.0s, 1056
effective words/s', 'datetime': '2026-03-08T17:58:10.041612',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}

Entrenamiento incremental completado
```

> **Nota:** No es posible continuar el entrenamiento de modelos cargados con `KeyedVectors.load_word2vec_format()`, ya que estos no contienen la información del árbol de vocabulario necesaria para el entrenamiento.

10. Cálculo de la Pérdida (Loss) durante el Entrenamiento

Podemos monitorear la pérdida durante el entrenamiento activando el parámetro `compute_loss`. Esto nos ayuda a verificar que el modelo está aprendiendo.

```python
# Entrenar con cálculo de pérdida activado
modelo_con_loss = gensim.models.Word2Vec(
    oraciones,
    min_count=1,
    compute_loss=True,
    hs=0,
    sg=1,   # Usamos Skip-gram
    seed=42,
)

# Obtener el valor de la pérdida del entrenamiento
perdida = modelo_con_loss.get_latest_training_loss()
print(f"Pérdida del entrenamiento: {perdida:.4f}")
```

```
2026-03-08 17:58:20,887 : INFO : collecting all words and their counts
2026-03-08 17:58:20,888 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 17:58:20,918 : INFO : collected 6981 word types from a
corpus of 58152 raw words and 300 sentences
2026-03-08 17:58:20,918 : INFO : Creating a fresh vocabulary
2026-03-08 17:58:20,924 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=1 retains 6981 unique words (100.00% of original
6981, drops 0)', 'datetime': '2026-03-08T17:58:20.924596', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:58:20,925 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=1 leaves 58152 word corpus (100.00% of original
58152, drops 0)', 'datetime': '2026-03-08T17:58:20.925021', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
```

```
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:58:20,932 : INFO : deleting the raw counts dictionary of
6981 items
2026-03-08 17:58:20,932 : INFO : sample=0.001 downsamples 43 most-
common words
2026-03-08 17:58:20,932 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 45723.4541622429 word corpus (78.6%% of
prior 58152)', 'datetime': '2026-03-08T17:58:20.932971', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 17:58:20,946 : INFO : estimated required memory for 6981
words and 100 dimensions: 9075300 bytes
2026-03-08 17:58:20,946 : INFO : resetting layer weights
2026-03-08 17:58:20,949 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T17:58:20.949139',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 17:58:20,949 : INFO : Word2Vec lifecycle event {'msg':
'training model with 3 workers on 6981 vocabulary and 100 features,
using sg=1 hs=0 sample=0.001 negative=5 window=5 shrink_windows=True',
'datetime': '2026-03-08T17:58:20.949350', 'gensim': '4.4.0', 'python':
'3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0
(clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit',
'event': 'train'}
2026-03-08 17:58:21,013 : INFO : EPOCH 0: training on 58152 raw words
(45668 effective words) took 0.1s, 721256 effective words/s
2026-03-08 17:58:21,073 : INFO : EPOCH 1: training on 58152 raw words
(45883 effective words) took 0.1s, 779208 effective words/s
2026-03-08 17:58:21,134 : INFO : EPOCH 2: training on 58152 raw words
(45691 effective words) took 0.1s, 759563 effective words/s
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
2026-03-08 17:58:21,197 : INFO : EPOCH 3: training on 58152 raw words
(45752 effective words) took 0.1s, 726790 effective words/s
2026-03-08 17:58:21,260 : INFO : EPOCH 4: training on 58152 raw words
(45808 effective words) took 0.1s, 737594 effective words/s
2026-03-08 17:58:21,260 : INFO : Word2Vec lifecycle event {'msg':
'training on 290760 raw words (228802 effective words) took 0.3s,
734828 effective words/s', 'datetime': '2026-03-08T17:58:21.260887',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 17:58:21,261 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=6981, vector_size=100, alpha=0.025>', 'datetime':
'2026-03-08T17:58:21.261082', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}
```

```
Pérdida del entrenamiento: 1369123.7500
```

11. Visualización de los Embeddings

Una forma intuitiva de entender lo que Word2Vec ha aprendido es **visualizar los vectores en 2D** usando la técnica t-SNE (*t-Distributed Stochastic Neighbor Embedding*).

En una buena visualización deberías poder observar:

- **Agrupaciones semánticas:** palabras como "perro", "gato", "vaca" aparecen juntas
- **Agrupaciones sintácticas:** palabras como "correr", "corriendo" están cerca
- **Relaciones vectoriales:** `vec(rey) - vec(hombre) ≈ vec(reina) - vec(mujer)`

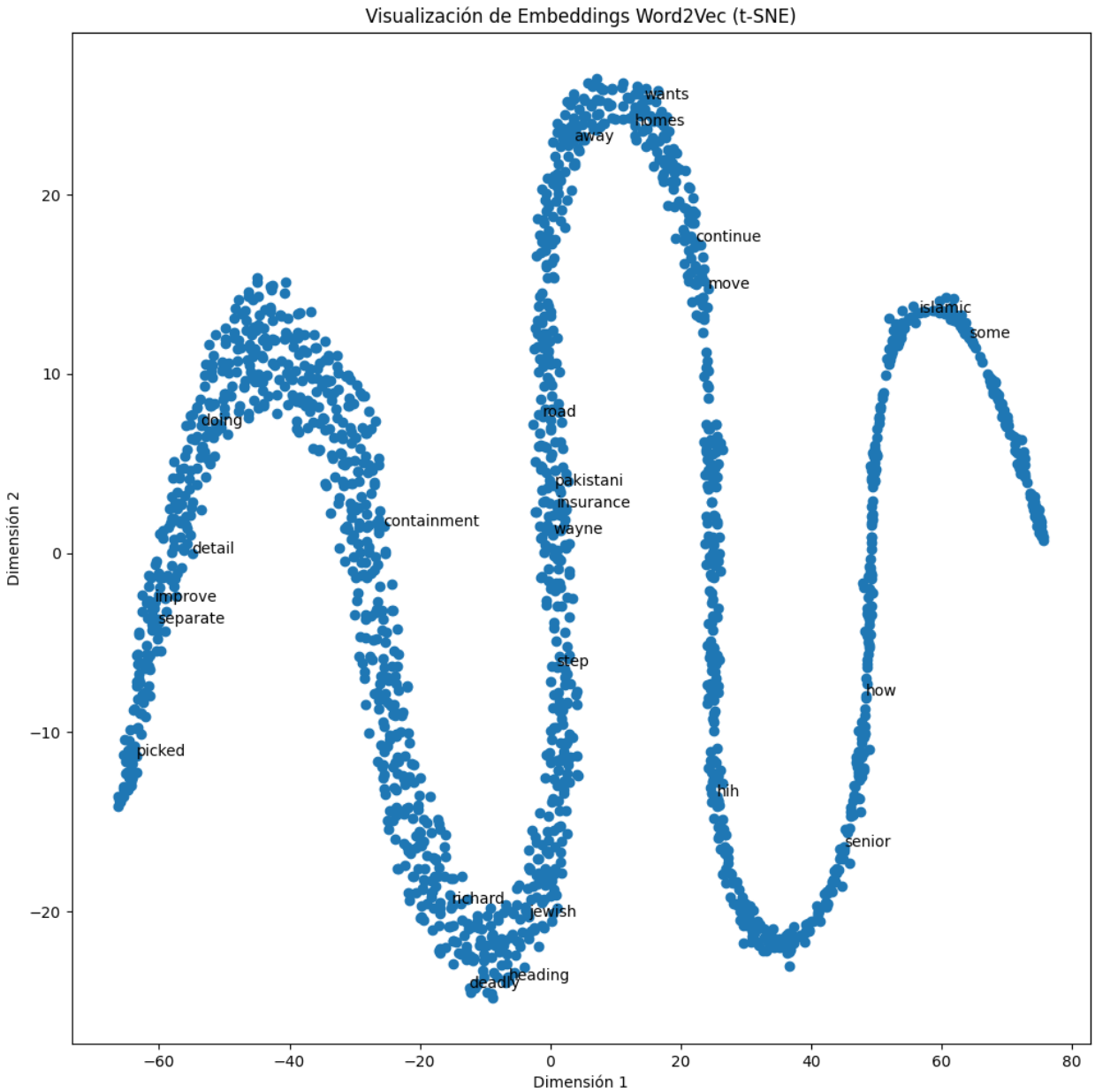
  Nota: El modelo está entrenado con un corpus pequeño, por lo que las relaciones pueden no ser tan claras.

```
from sklearn.manifold import TSNE                    # Reducción final
a 2D
import numpy as np


def reducir_dimensiones(modelo):
    """Reduce los embeddings a 2 dimensiones usando t-SNE."""
    num_dimensiones = 2

    # Extraer los vectores y las etiquetas como arrays de NumPy
    vectores = np.asarray(modelo.wv.vectors)
    etiquetas = np.asarray(modelo.wv.index_to_key)

    # Reducir dimensionalidad con t-SNE
    tsne = TSNE(n_components=num_dimensiones, random_state=0)
    vectores = tsne.fit_transform(vectores)

    x_vals = [v[0] for v in vectores]
    y_vals = [v[1] for v in vectores]
    return x_vals, y_vals, etiquetas


x_vals, y_vals, etiquetas = reducir_dimensiones(modelo)

def graficar_con_plotly(x_vals, y_vals, etiquetas, en_notebook=True):
    """Visualiza los embeddings usando Plotly (interactivo)."""
    from plotly.offline import init_notebook_mode, iplot, plot
    import plotly.graph_objs as go

    traza = go.Scatter(x=x_vals, y=y_vals, mode='text',
text=etiquetas)
    datos = [traza]

    if en_notebook:
```

```
        init_notebook_mode(connected=True)
        iplot(datos, filename='embedding-palabras')
    else:
        plot(datos, filename='embedding-palabras.html')


def graficar_con_matplotlib(x_vals, y_vals, etiquetas):
    """Visualiza los embeddings usando Matplotlib (estático)."""
    import matplotlib.pyplot as plt
    import random

    random.seed(0)

    plt.figure(figsize=(12, 12))
    plt.scatter(x_vals, y_vals)
    plt.title('Visualización de Embeddings Word2Vec (t-SNE)')
    plt.xlabel('Dimensión 1')
    plt.ylabel('Dimensión 2')

    # Etiquetar 25 puntos seleccionados al azar
    indices = list(range(len(etiquetas)))
    indices_seleccionados = random.sample(indices, 25)
    for i in indices_seleccionados:
        plt.annotate(etiquetas[i], (x_vals[i], y_vals[i]))

    plt.show()


def _nbformat_disponible(min_major=4, min_minor=2):
    try:
        import nbformat
        partes = nbformat.__version__.split('.')
        major = int(partes[0])
        minor = int(partes[1]) if len(partes) > 1 else 0
        return (major, minor) >= (min_major, min_minor)
    except Exception:
        return False


# Seleccionar la función de graficación apropiada
try:
    get_ipython()
    en_notebook = True
except Exception:
    en_notebook = False

if en_notebook and _nbformat_disponible():
    funcion_graficar = graficar_con_plotly
else:
    if en_notebook:
```

```
        print('nbformat>=4.2.0 no está disponible; usando Matplotlib
como alternativa.')
    funcion_graficar = graficar_con_matplotlib

funcion_graficar(x_vals, y_vals, etiquetas)
```

```
nbformat>=4.2.0 no está disponible; usando Matplotlib como
alternativa.
```



12. Resumen y Conceptos Clave

En este tutorial aprendimos:

Concepto	Descripción
Bolsa de Palabras	Representación simple que pierde orden y significado
Word2Vec	Genera embeddings que capturan relaciones semánticas
Skip-gram	Predice el contexto a partir de una palabra central
CBOW	Predice la palabra central a partir del contexto
Similitud coseno	Mide qué tan parecidos son dos vectores
t-SNE	Técnica para visualizar vectores de alta dimensión en 2D

Parámetros más importantes:

- `vector_size`: Dimensión de los embeddings (más grande = más expresivo, pero necesita más datos)
- `min_count`: Frecuencia mínima para incluir una palabra
- `sg`: 0 para CBOW, 1 para Skip-gram
- `workers`: Número de hilos para paralelizar el entrenamiento

Enlaces útiles:

- Documentación de Gensim Word2Vec
- Artículos originales de Word2Vec por Google
- Tutorial visual de Word2Vec (Jay Alammar)

Parte 3: Aplicación de Word2Vec al Dataset de Libros

Ahora que comprendemos cómo funciona Word2Vec, vamos a aplicar esta técnica al dataset de libros de LibraryThing que utilizamos en la Parte 1.

La idea es:

1. Tratar los conceptos (features) de cada libro como "palabras" y cada libro como una "oración"
2. Entrenar un modelo Word2Vec sobre estas "oraciones" para obtener embeddings de conceptos
3. Representar cada libro como el promedio de los embeddings de sus conceptos
4. Usar esta nueva representación para el modelo de recomendación y comparar con TF-IDF

Paso 1: Preparar los datos como "oraciones" de conceptos

Cada libro tiene un conjunto de conceptos de DBpedia asociados. Vamos a crear una lista de "oraciones" donde cada oración es la lista de nombres cortos de conceptos (featureShortname) de un libro.

```
# TODO: Crear una lista de oraciones a partir de los conceptos de cada libro
# Cada oración debe ser una lista de strings con los featureShortname de un libro
# Hint: Agrupe df_libros_concepto por DBbook_ItemID y recolecte los featureShortname

oraciones_libros = (
    df_libros_concepto
    .groupby('DBbook_ItemID')['featureShortname']
    .apply(list)
    .tolist()
)

print(f'Número de libros (oraciones): {len(oraciones_libros)}')
print(f'Ejemplo de oracion (primer libro): {oraciones_libros[0][:10]}...')
```

```
Número de libros (oraciones): 6164
Ejemplo de oracion (primer libro): ['/Diana_Gabaldon', '/Historical_novel', '/Communication100033020', '/Fiction106367107', '/LiteraryComposition106364329', '/Writing106362953', '/WrittenCommunication106349220', '/1992Novels', '/AmericanHistoricalNovels', '/HispanicAmericanNovels']...
```

Paso 2: Entrenar un modelo Word2Vec con los conceptos de los libros

Complete la celda siguiente entrenando un modelo Word2Vec con los parámetros que considere apropiados. Considere:

- `vector_size`: ¿cuántas dimensiones? (pruebe con 50 o 100)
- `window`: ¿qué tamaño de ventana? (los conceptos de un libro no tienen orden estricto)
- `min_count`: ¿frecuencia mínima?
- `sg`: ¿Skip-gram o CBOW?

```
import gensim.models

modelo_w2v = gensim.models.Word2Vec(
    sentences=oraciones_libros,
    vector_size=100,
    window=10,
    min_count=2,
    sg=1,
    workers=4
```

```
)

print(f'Vocabulario del modelo: {len(modelo_w2v.wv)} conceptos')
print(f'Dimensión de los embeddings: {modelo_w2v.wv.vector_size}')
```

```
2026-03-08 18:21:40,369 : INFO : collecting all words and their counts
2026-03-08 18:21:40,370 : INFO : PROGRESS: at sentence #0, processed 0
words, keeping 0 word types
2026-03-08 18:21:40,379 : INFO : collected 12245 word types from a
corpus of 102803 raw words and 6164 sentences
2026-03-08 18:21:40,380 : INFO : Creating a fresh vocabulary
2026-03-08 18:21:40,387 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=2 retains 5814 unique words (47.48% of original
12245, drops 6431)', 'datetime': '2026-03-08T18:21:40.387385',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 18:21:40,387 : INFO : Word2Vec lifecycle event {'msg':
'effective_min_count=2 leaves 96372 word corpus (93.74% of original
102803, drops 6431)', 'datetime': '2026-03-08T18:21:40.387878',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 18:21:40,394 : INFO : deleting the raw counts dictionary of
12245 items
2026-03-08 18:21:40,395 : INFO : sample=0.001 downsamples 35 most-
common words
2026-03-08 18:21:40,395 : INFO : Word2Vec lifecycle event {'msg':
'downsampling leaves estimated 63065.3004991101 word corpus (65.4%% of
prior 96372)', 'datetime': '2026-03-08T18:21:40.395579', 'gensim':
'4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50)
[Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-
arm-64bit', 'event': 'prepare_vocab'}
2026-03-08 18:21:40,407 : INFO : estimated required memory for 5814
words and 100 dimensions: 7558200 bytes
2026-03-08 18:21:40,408 : INFO : resetting layer weights
2026-03-08 18:21:40,409 : INFO : Word2Vec lifecycle event {'update':
False, 'trim_rule': 'None', 'datetime': '2026-03-08T18:21:40.409880',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'build_vocab'}
2026-03-08 18:21:40,410 : INFO : Word2Vec lifecycle event {'msg':
'training model with 4 workers on 5814 vocabulary and 100 features,
using sg=1 hs=0 sample=0.001 negative=5 window=10
shrink_windows=True', 'datetime': '2026-03-08T18:21:40.410066',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 18:21:40,477 : INFO : EPOCH 0: training on 102803 raw words
(63166 effective words) took 0.1s, 970725 effective words/s
```

```
2026-03-08 18:21:40,540 : INFO : EPOCH 1: training on 102803 raw words
(63017 effective words) took 0.1s, 1025218 effective words/s
2026-03-08 18:21:40,607 : INFO : EPOCH 2: training on 102803 raw words
(63093 effective words) took 0.1s, 978049 effective words/s
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
2026-03-08 18:21:40,672 : INFO : EPOCH 3: training on 102803 raw words
(63111 effective words) took 0.1s, 999793 effective words/s
2026-03-08 18:21:40,733 : INFO : EPOCH 4: training on 102803 raw words
(62951 effective words) took 0.1s, 1068500 effective words/s
2026-03-08 18:21:40,733 : INFO : Word2Vec lifecycle event {'msg':
'training on 514015 raw words (315338 effective words) took 0.3s,
973717 effective words/s', 'datetime': '2026-03-08T18:21:40.733912',
'gensim': '4.4.0', 'python': '3.11.9 (v3.11.9:de54cf5be3, Apr  2 2024,
07:12:50) [Clang 13.0.0 (clang-1300.0.29.30)]', 'platform': 'macOS-
15.5-arm64-arm-64bit', 'event': 'train'}
2026-03-08 18:21:40,734 : INFO : Word2Vec lifecycle event {'params':
'Word2Vec<vocab=5814, vector_size=100, alpha=0.025>', 'datetime':
'2026-03-08T18:21:40.734150', 'gensim': '4.4.0', 'python': '3.11.9
(v3.11.9:de54cf5be3, Apr  2 2024, 07:12:50) [Clang 13.0.0 (clang-
1300.0.29.30)]', 'platform': 'macOS-15.5-arm64-arm-64bit', 'event':
'created'}

Vocabulario del modelo: 5814 conceptos
Dimensión de los embeddings: 100
```

Paso 3: Explorar los embeddings aprendidos

Verifique que los embeddings tienen sentido buscando conceptos similares.

```python
# TODO: Explore los embeddings aprendidos
# Pruebe buscando los conceptos más similares a algún concepto del
vocabulario
# Ejemplo: modelo_w2v.wv.most_similar('Novel', topn=10)

# Ver algunos conceptos del vocabulario
print("Primeros 20 conceptos del vocabulario:")
print(modelo_w2v.wv.index_to_key[:20])

# Probar similitud con algunos conceptos
conceptos_prueba = [
    '/Fantasy_literature',
    '/Novel',
    '/Lynn_Flewelling',
    '/Category:American_fantasy_novels'
]

for concepto in conceptos_prueba:
    if concepto in modelo_w2v.wv:
        print(f"\nConceptos más similares a {concepto}:")
```

```
        similares = modelo_w2v.wv.most_similar(concepto, topn=10)
        for sim, score in similares:
            print(f"  {sim:40s} {score:.4f}")
    else:
        print(f"\nEl concepto {concepto} no está en el vocabulario del
modelo.")
```

```
Primeros 20 conceptos del vocabulario:
['/Abstraction100002137', '/Communication100033020',
'/WrittenCommunication106349220', '/Writing106362953',
'/LiteraryComposition106364329', '/Fiction106367107',
'/Novel106367879', '/PhysicalEntity100001930', '/Whole100003553',
'/Object100002684', '/Artifact100021939', '/Product104007894',
'/Creation103129123', '/Publication106589574', '/Work104599396',
'/Book106410904', '/Science_fiction', '/Novel', '/Fantasy_literature',
'/PsychologicalFeature100023100']

Conceptos más similares a /Fantasy_literature:
  /High_fantasy                              0.9489
  /Category:Discworld_books                  0.9372
  /Margaret_Weis                             0.9342
  /Terry_Pratchett                           0.9336
  /Category:1990s_fantasy_novels             0.9334
  /Glen_Cook                                 0.9313
  /Tracy_Hickman                             0.9308
  /Mercedes_Lackey                           0.9177
  /Category:Novels_by_Raymond_E._Feist       0.9153
  /Category:1980s_fantasy_novels             0.9138

Conceptos más similares a /Novel:
  /Category:Novels_set_in_Mississippi        0.9367
  /Category:Novels_set_in_Paris              0.9335
  /Category:Novels_about_music               0.9309
  /Category:Edgar_Award_winning_works        0.9295
  /Category:Novels_set_in_Georgia_(U.S._state) 0.9232
  /Historical_fiction                        0.9209
  /Category:National_Book_Award_for_Fiction_winning_works 0.9204
  /Category:Pulitzer_Prize_for_Fiction_winning_works 0.9166
  /Category:American_bildungsromans          0.9163
  /Category:Novels_set_in_Scotland           0.9135

Conceptos más similares a /Lynn_Flewelling:
  /Ray_Bradbury                              0.9830
  /Steven_Brust                              0.9822
  /Category:The_Death_Gate_Cycle_novels      0.9810
  /Category:Novels_by_Guy_Gavriel_Kay        0.9764
  /Category:Garrett_P.I.                     0.9760
  /Guy_Gavriel_Kay                           0.9746
  /Category:The_Sword_of_Truth_books         0.9727
  /Category:Nightrunner_series               0.9727
```

```
  /Terry_Goodkind                          0.9718
  /EmelaneseBooks                          0.9710

Conceptos más similares a /Category:American_fantasy_novels:
  /AmericanFantasyNovels                   0.9382
  /Margaret_Weis                           0.8926
  /Fantasy_literature                      0.8861
  /Tracy_Hickman                           0.8752
  /High_fantasy                            0.8628
  /Category:Novels_by_Raymond_E._Feist     0.8602
  /Raymond_E._Feist                        0.8579
  /Mercedes_Lackey                         0.8539
  /CoreDragonlanceNovels                   0.8506
  /NovelsByRaymondE.Feist                  0.8492
```

Paso 4: Representar cada libro como el promedio de sus embeddings

Para obtener un vector por libro, calculamos el **promedio** de los embeddings de todos sus conceptos. Esto nos da una representación densa de cada libro.

```
# TODO: Crear la matriz de representación de libros usando Word2Vec
# Para cada libro, calcule el promedio de los embeddings de sus
conceptos

def obtener_embedding_libro(item_id, modelo_wv, df_libros_concepto):
    """Calcula el embedding promedio de un libro a partir de sus
conceptos."""
    conceptos = df_libros_concepto.loc[
        df_libros_concepto.DBbook_ItemID == item_id,
'featureShortname'
    ].unique()
    vectores = []
    for c in conceptos:
        if c in modelo_wv.key_to_index:
            vectores.append(modelo_wv[c])
    if vectores:
        return np.mean(vectores, axis=0)
    else:
        return np.zeros(modelo_wv.vector_size)


# Construir la matriz de embeddings para todos los libros
ids_libros = df_libros['DBbook_ItemID'].values
embeddings_libros = np.array([
    obtener_embedding_libro(item_id, modelo_w2v.wv,
df_libros_concepto)
    for item_id in ids_libros
])
```

```
df_matriz_w2v = pd.DataFrame(embeddings_libros, index=ids_libros)
print(f'Forma de la matriz Word2Vec: {df_matriz_w2v.shape}')
df_matriz_w2v.head()
```

```
Forma de la matriz Word2Vec: (6164, 100)

            0         1         2         3         4         5
6   \
508  -0.098560  0.138031  0.243491  0.081686  0.119182 -0.305973 -
0.058626
802  -0.325940  0.099686 -0.037579  0.247516 -0.000163 -0.476742
0.218219
889  -0.333988  0.107614 -0.030140  0.162583  0.084907 -0.398769
0.170931
1367 -0.169047 -0.040838  0.158462  0.284439  0.151414 -0.389324 -
0.103715
1736 -0.131462 -0.001942  0.260018  0.194588  0.108601 -0.431743 -
0.094185

            7         8         9   ...        90        91        92
\
508   0.337129 -0.009836 -0.226975  ...  0.140951  0.040052 -0.130480

802   0.411616 -0.040754 -0.138444  ...  0.115256  0.109521 -0.005725

889   0.408955  0.046059 -0.079240  ...  0.141138  0.123450  0.009039

1367  0.294940  0.043212 -0.184740  ...  0.119804  0.234050 -0.114427

1736  0.380638 -0.028265 -0.218275  ...  0.047648  0.199831 -0.089538


            93        94        95        96        97        98
99
508   0.142439  0.045500  0.053055  0.192001  0.024573  0.145366
0.059366
802  -0.174305  0.503438  0.228449  0.288960 -0.125760  0.053835 -
0.072020
889  -0.214991  0.412675  0.195691  0.190790 -0.152744  0.041154 -
0.015747
1367 -0.106222  0.309327  0.058145  0.488252  0.077586  0.170261
0.025790
1736  0.051494  0.253276  0.024903  0.470545  0.035331  0.205348
0.089933

[5 rows x 100 columns]
```

Paso 5: Entrenar el modelo KNN con representación Word2Vec

Complete la celda siguiente para entrenar un clasificador KNN usando la representación Word2Vec en lugar de TF-IDF, y evalúe los resultados para un usuario de ejemplo.

```python
# TODO: Repita el proceso de clasificación usando la representación Word2Vec
# 1. Seleccione un usuario del conjunto de test
# 2. Obtenga sus items de entrenamiento y test
# 3. Asocie la representación Word2Vec (df_matriz_w2v) en lugar de TF-IDF
# 4. Entrene un KNeighborsClassifier
# 5. Evalúe con confusion_matrix y precision_recall_fscore_support

# 1. Seleccionar un usuario de ejemplo
usuario_ejemplo = df_all_interactions_test.iloc[0]['DBbook_userID']

# 2. Obtener sus items de train y test
df_train_w2v = df_all_interactions_train.loc[
    df_all_interactions_train.DBbook_userID == usuario_ejemplo,
    ['DBbook_ItemID', 'class']
].merge(
    df_matriz_w2v,
    how='left',
    left_on='DBbook_ItemID',
    right_index=True
)

df_test_w2v = df_all_interactions_test.loc[
    df_all_interactions_test.DBbook_userID == usuario_ejemplo,
    ['DBbook_ItemID', 'class']
].merge(
    df_matriz_w2v,
    how='left',
    left_on='DBbook_ItemID',
    right_index=True
)

# 3. Definir features Word2Vec
features_w2v = df_matriz_w2v.columns

# 4. Entrenar el clasificador KNN
knn_w2v = KNeighborsClassifier(n_neighbors=3)
knn_w2v.fit(df_train_w2v[features_w2v], df_train_w2v['class'])


# 5. Predecir sobre test
df_test_w2v['predict'] = knn_w2v.predict(df_test_w2v[features_w2v])

# 6. Evaluar
tn, fp, fn, tp = confusion_matrix(
    df_test_w2v['class'],
    df_test_w2v['predict'],
    labels=[False, True]
).ravel()
```

```
print('Confusion matrix (tn, fp, fn, tp):', tn, fp, fn, tp)
print('Precision, Recall, F1, Support:')
print(
    precision_recall_fscore_support(
        df_test_w2v['class'],
        df_test_w2v['predict'],
        pos_label=True,
        average='binary'
    )
)

# 7. Ver resultados con nombres de libros
df_test_w2v[['DBbook_ItemID', 'class', 'predict']].merge(
    df_libros,
    how='left',
    on='DBbook_ItemID'
)
```

```
Confusion matrix (tn, fp, fn, tp): 0 2 0 2
Precision, Recall, F1, Support:
(0.5, 1.0, 0.6666666666666666, None)
```

```
   DBbook_ItemID  class  predict                                name
0           1971  False     True                   A Virtuous Woman
1           7682   True     True                    A Painted House
2           5842   True     True  The Unbearable Lightness of Being
3           5021  False     True                 The Dream of Scipio
```

Comparación: TF-IDF vs Word2Vec para Filtrado por Contenido

Ahora que ha implementado ambos enfoques, compare los resultados.

Ejercicio: Evaluación comparativa

1. Ejecute el modelo KNN con representación TF-IDF y Word2Vec para al menos 50 usuarios
2. Calcule el precision, recall y f-score promedio para cada representación
3. Compare los resultados y discuta cuál representación funciona mejor y por qué

```
# TODO: Implemente la evaluación comparativa entre TF-IDF y Word2Vec
# Calcule las métricas promedio para ambos modelos sobre múltiples
usuarios

def evaluar_representacion_knn(df_repr, nombre_modelo, usuarios_eval,
k=3):
    """
```

```
    Evalúa KNN para una representación de ítems dada (TF-IDF o
Word2Vec)
    sobre un conjunto de usuarios.
    """
    resultados_usuario = []
    predicciones_globales = []

    features = df_repr.columns

    for user in usuarios_eval:
        # train del usuario
        df_train_user = df_all_interactions_train.loc[
            df_all_interactions_train['DBbook_userID'] == user,
            ['DBbook_ItemID', 'class']
        ].merge(
            df_repr,
            how='left',
            left_on='DBbook_ItemID',
            right_index=True
        )

        # test del usuario
        df_test_user = df_all_interactions_test.loc[
            df_all_interactions_test['DBbook_userID'] == user,
            ['DBbook_ItemID', 'class']
        ].merge(
            df_repr,
            how='left',
            left_on='DBbook_ItemID',
            right_index=True
        )

        # filtros mínimos
        if df_train_user.empty or df_test_user.empty:
            continue
        if len(df_train_user) < k:
            continue
        if df_train_user['class'].nunique() < 2:
            continue

        X_train = df_train_user[features]
        y_train = df_train_user['class']
        X_test = df_test_user[features]
        y_test = df_test_user['class']

        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)

        # métricas por usuario
```

```
        precision, recall, f1, support =
precision_recall_fscore_support(
            y_test, y_pred, pos_label=True, average='binary',
zero_division=0
        )

        resultados_usuario.append({
            'modelo': nombre_modelo,
            'DBbook_userID': user,
            'precision': precision,
            'recall': recall,
            'f1': f1,
            'n_test_items': len(df_test_user)
        })

        # predicciones globales
        predicciones_globales.append(pd.DataFrame({
            'modelo': nombre_modelo,
            'DBbook_userID': user,
            'class_real': y_test.values,
            'class_pred': y_pred
        }))

    df_resultados_usuario = pd.DataFrame(resultados_usuario)

    if predicciones_globales:
        df_pred_global = pd.concat(predicciones_globales,
ignore_index=True)
        precision_g, recall_g, f1_g, support_g =
precision_recall_fscore_support(
            df_pred_global['class_real'],
            df_pred_global['class_pred'],
            pos_label=True,
            average='binary',
            zero_division=0
        )
    else:
        df_pred_global = pd.DataFrame()
        precision_g, recall_g, f1_g, support_g = np.nan, np.nan,
np.nan, np.nan

    resumen = pd.DataFrame([{
        'modelo': nombre_modelo,
        'k': k,
        'usuarios_evaluados':
df_resultados_usuario['DBbook_userID'].nunique() if not
df_resultados_usuario.empty else 0,
        'predicciones_totales': len(df_pred_global),
        'precision_promedio_usuarios':
df_resultados_usuario['precision'].mean() if not
```

```
df_resultados_usuario.empty else np.nan,
        'recall_promedio_usuarios':
df_resultados_usuario['recall'].mean() if not
df_resultados_usuario.empty else np.nan,
        'f1_promedio_usuarios': df_resultados_usuario['f1'].mean() if
not df_resultados_usuario.empty else np.nan,
        'precision_global': precision_g,
        'recall_global': recall_g,
        'f1_global': f1_g
    }])

    return df_resultados_usuario, df_pred_global, resumen
```

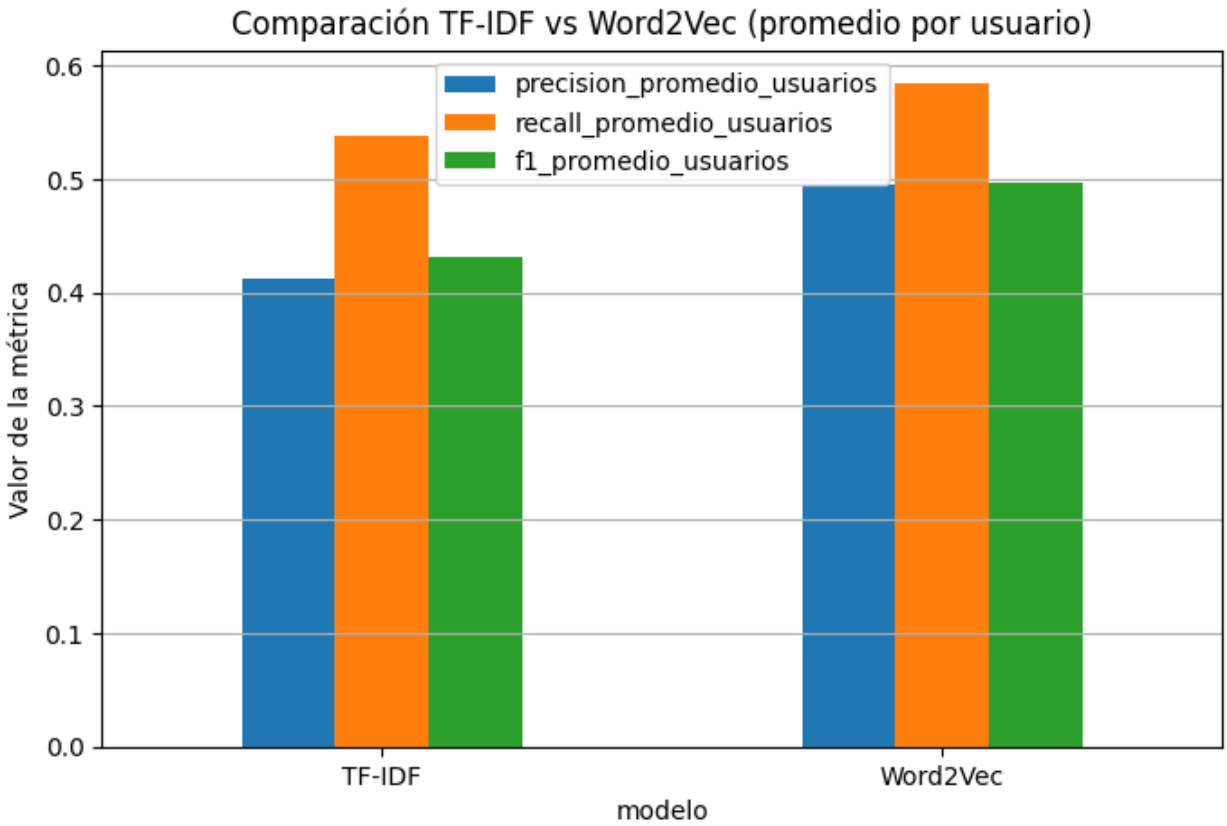
```
usuarios_eval = df_conteos_usuario_train_test.nlargest(500,
'test_count').index
len(usuarios_eval)
```

```
500
```

```
# TF-IDF
res_user_tfidf, pred_tfidf, resumen_tfidf =
evaluar_representacion_knn(
    df_repr=df_matriz_tf_idf,
    nombre_modelo='TF-IDF',
    usuarios_eval=usuarios_eval,
    k=3
)

# Word2Vec
res_user_w2v, pred_w2v, resumen_w2v = evaluar_representacion_knn(
    df_repr=df_matriz_w2v,
    nombre_modelo='Word2Vec',
    usuarios_eval=usuarios_eval,
    k=3
)

comparacion = pd.concat([resumen_tfidf, resumen_w2v],
ignore_index=True)
comparacion
```

```
     modelo  k  usuarios_evaluados  predicciones_totales  \
0    TF-IDF  3                 493                  2375
1  Word2Vec  3                 493                  2375

   precision_promedio_usuarios  recall_promedio_usuarios  \
0                     0.412204                  0.537728
1                     0.494692                  0.584212

   f1_promedio_usuarios  precision_global  recall_global  f1_global
0              0.431376          0.587314       0.607287   0.597134
1              0.496735          0.573688       0.646154   0.607768
```

```
comparacion_plot = comparacion.set_index('modelo')[
    ['precision_promedio_usuarios', 'recall_promedio_usuarios',
'f1_promedio_usuarios']
]

comparacion_plot.plot(kind='bar', figsize=(8,5))
plt.title('Comparación TF-IDF vs Word2Vec (promedio por usuario)')
plt.ylabel('Valor de la métrica')
plt.xticks(rotation=0)
plt.grid(axis='y')
plt.show()
```



Conclusión

Se evaluaron los modelos KNN usando las representaciones TF-IDF y Word2Vec sobre 493 usuarios del conjunto de prueba, con un total de 2375 predicciones. Los resultados muestran que Word2Vec obtuvo mejores métricas promedio por usuario: precision = 0.4947, recall = 0.5842 y F1 = 0.4967, frente a TF-IDF con precision = 0.4122, recall = 0.5377 y F1 = 0.4314.

En la evaluación global, TF-IDF logró una precision ligeramente mayor (0.5873 vs 0.5737), pero Word2Vec obtuvo mejor recall (0.6462 vs 0.6073) y mejor F1-score (0.6078 vs 0.5971). Esto sugiere que Word2Vec recupera mejor los ítems relevantes y ofrece un mejor balance general entre precisión y recall.

Una posible explicación es que Word2Vec captura relaciones semánticas entre conceptos relacionados, incluso cuando los libros no comparten exactamente las mismas features de DBpedia. En cambio, TF-IDF depende de coincidencias exactas entre conceptos. Por esta razón, la representación densa de Word2Vec parece más adecuada para este problema de filtrado por contenido.

Ejercicio de reflexión: Comparación de estrategias

Responda las siguientes preguntas basándose en los resultados obtenidos:

1. ¿Cuál representación obtuvo mejores resultados? ¿Por qué cree que es así?
2. ¿Cuáles son las ventajas computacionales de cada enfoque? (dimensionalidad, tiempo de entrenamiento, memoria)
3. ¿En qué escenarios sería más apropiado usar TF-IDF vs Word2Vec?
4. ¿Cómo se podrían aplicar estas representaciones en un esquema de **Filtrado Colaborativo**? (Hint: piense en Item2Vec, donde las secuencias de ítems consumidos por un usuario se tratan como oraciones)

Respuesta 1:

En los experimentos realizados, la representación Word2Vec obtuvo mejores resultados que TF-IDF. En el promedio por usuario, Word2Vec superó a TF-IDF en precision, recall y f1-score. Además, en la evaluación global también consiguió mejor recall y mejor f1, aunque TF-IDF tuvo una precision global ligeramente superior. Esto puede explicarse porque Word2Vec captura relaciones semánticas entre conceptos que aparecen en contextos similares, incluso si no coinciden exactamente. En cambio, TF-IDF depende de la coincidencia exacta entre features. Por eso, Word2Vec puede generalizar mejor entre libros relacionados temáticamente.

Respuesta 2:

TF-IDF tiene como ventaja que su construcción es relativamente directa e interpretable, pero su principal desventaja computacional es la alta dimensionalidad: genera miles de columnas y una matriz muy dispersa, lo que implica mayor uso de memoria y puede hacer más costoso el cálculo de similitudes o el entrenamiento de modelos. Word2Vec, en cambio, requiere un paso adicional de entrenamiento para aprender los embeddings, pero una vez entrenado produce representaciones densas de baja dimensionalidad, por ejemplo de 100 dimensiones, lo cual reduce memoria y hace más eficientes operaciones posteriores como KNN. En resumen, TF-IDF es más simple pero más grande y disperso; Word2Vec requiere más esfuerzo inicial, pero produce representaciones más compactas.

Respuesta 3:

TF-IDF es más apropiado cuando se necesita interpretabilidad y cuando las coincidencias exactas entre conceptos son importantes. También puede ser útil en datasets pequeños o cuando se desea saber explícitamente qué atributos explican una recomendación. Word2Vec es más apropiado cuando interesa capturar similitud semántica entre conceptos relacionados, reducir la dimensionalidad y obtener representaciones más compactas. Es especialmente útil cuando dos libros pueden ser parecidos aunque no compartan exactamente los mismos conceptos de DBpedia.

Respuesta 4:

Estas representaciones también pueden aplicarse en filtrado colaborativo. En lugar de tratar los conceptos de un libro como palabras, se pueden tratar los ítems consumidos por un usuario como una oración. Así, las secuencias de ítems interactuados por cada usuario sirven como corpus para entrenar embeddings de ítems. Esta idea corresponde a Item2Vec. En ese caso, ítems que suelen aparecer juntos en los historiales de consumo de muchos usuarios tendrían embeddings similares. Luego esos embeddings podrían usarse para recomendar ítems cercanos a los ya consumidos por el usuario, o como entrada para modelos híbridos que combinen filtrado colaborativo y contenido.